

# TeamBook de romerproblem

## Índice

|                                                |           |
|------------------------------------------------|-----------|
| <b>1. Math</b>                                 | <b>1</b>  |
| 1.1. matrizFibo . . . . .                      | 1         |
| <b>2. Ultimos</b>                              | <b>1</b>  |
| 2.1. FordFulkerson . . . . .                   | 1         |
| 2.2. HoprofKarp . . . . .                      | 1         |
| 2.3. centroid . . . . .                        | 2         |
| 2.4. dinitz . . . . .                          | 4         |
| 2.5. edmonkarp . . . . .                       | 5         |
| 2.6. extended . . . . .                        | 5         |
| 2.7. fastgcd . . . . .                         | 6         |
| 2.8. fft . . . . .                             | 6         |
| 2.9. gauss_algebra_lineal . . . . .            | 7         |
| 2.10. geometrynotes . . . . .                  | 7         |
| 2.11. hld . . . . .                            | 7         |
| 2.12. iterativequerysegment . . . . .          | 9         |
| 2.13. khun . . . . .                           | 9         |
| 2.14. latticepoint . . . . .                   | 10        |
| 2.15. lcs . . . . .                            | 10        |
| 2.16. linearsieve . . . . .                    | 10        |
| 2.17. lis . . . . .                            | 11        |
| 2.18. nearestVecino . . . . .                  | 11        |
| 2.19. pointintpoligon . . . . .                | 12        |
| 2.20. starsandbars . . . . .                   | 13        |
| 2.21. suffixarray . . . . .                    | 13        |
| 2.22. trie . . . . .                           | 13        |
| 2.23. trie2 . . . . .                          | 15        |
| 2.24. twoSegmentIntersec . . . . .             | 15        |
| <b>3. Updates</b>                              | <b>16</b> |
| 3.1. 2-sat . . . . .                           | 16        |
| 3.2. ahocorasick . . . . .                     | 17        |
| 3.3. componentes_fuertemente_conexos . . . . . | 18        |
| 3.4. floyd_warshall . . . . .                  | 18        |
| 3.5. lazy . . . . .                            | 18        |
| <b>4. Bits</b>                                 | <b>19</b> |
| 4.1. SoSDP . . . . .                           | 19        |

|                                          |           |
|------------------------------------------|-----------|
| <b>5. Miscelanea</b>                     | <b>19</b> |
| 5.1. Plantilla . . . . .                 | 19        |
| <b>6. Datastructures</b>                 | <b>20</b> |
| 6.1. LowestCommonAncestor . . . . .      | 20        |
| 6.2. Mo . . . . .                        | 20        |
| 6.3. QueueMin . . . . .                  | 22        |
| 6.4. SegmentTree . . . . .               | 23        |
| 6.5. SparseTable . . . . .               | 24        |
| 6.6. UnionFind . . . . .                 | 24        |
| 6.7. UnionFind2D . . . . .               | 24        |
| 6.8. fenwickTree . . . . .               | 25        |
| 6.9. sqrt . . . . .                      | 25        |
| 6.10. sqrtUpdateinSqrt . . . . .         | 26        |
| <b>7. Strings</b>                        | <b>27</b> |
| 7.1. KMP . . . . .                       | 27        |
| 7.2. hashing . . . . .                   | 27        |
| 7.3. hashing2d . . . . .                 | 28        |
| 7.4. hashingmersense . . . . .           | 29        |
| 7.5. manacher . . . . .                  | 29        |
| 7.6. zfunction . . . . .                 | 30        |
| <b>8. Geometry</b>                       | <b>30</b> |
| 8.1. KDTree . . . . .                    | 30        |
| 8.2. Point . . . . .                     | 31        |
| 8.3. convexHull . . . . .                | 32        |
| 8.4. pointinpoligon . . . . .            | 33        |
| <b>9. Primalidad</b>                     | <b>34</b> |
| 9.1. RabinMiller . . . . .               | 34        |
| 9.2. pollardRho . . . . .                | 35        |
| <b>10. Graphs</b>                        | <b>36</b> |
| 10.1. articulationpoint . . . . .        | 36        |
| 10.2. bridges . . . . .                  | 36        |
| 10.3. dijkstra . . . . .                 | 37        |
| 10.4. dinic . . . . .                    | 37        |
| 10.5. eulerianpath . . . . .             | 38        |
| 10.6. isDag . . . . .                    | 39        |
| 10.7. isDagv2 . . . . .                  | 40        |
| 10.8. khunMaxbipartiteMatching . . . . . | 40        |
| 10.9. lcabinarylifitin . . . . .         | 41        |
| 10.10. toposort . . . . .                | 41        |

|                                |           |
|--------------------------------|-----------|
| <b>11.Overloads</b>            | <b>42</b> |
| 11.1. pq . . . . .             | 42        |
| 11.2. set . . . . .            | 42        |
| <b>12.Dp</b>                   | <b>42</b> |
| 12.1. covexHullTrick . . . . . | 42        |
| 12.2. list . . . . .           | 43        |

# 1. Math

## 1.1. matrizFibo

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const ll MOD = 1'000'000'007;
6
7 struct Matrix {
8     ll mat[2][2];
9 };
10
11 Matrix operator*(const Matrix &a, const Matrix &b){
12     Matrix c{};
13     for (int i = 0; i < 2; i++)
14         for (int j = 0; j < 2; j++) {
15             __int128 tmp = 0;
16             for (int k = 0; k < 2; k++)
17                 tmp += (__int128)a.mat[i][k] * b.mat[k][j];
18             c.mat[i][j] = (ll)(tmp % MOD);
19         }
20     return c;
21 }
22
23 Matrix matpow(Matrix base, ll n) {
24     Matrix ans{{ {1, 0}, {0, 1} }};
25     while (n) {
26         if (n & 1) ans = ans * base;
27         base = base * base;
28         n >>= 1;
29     }
30     return ans;
31 }
32
33 ll fib(ll n) {
34     if (n == 0) return 0;
35     Matrix base{{ {1,1}, {1,0} }};
36     return matpow(base, n).mat[0][1];
37 }
38
39 int main(){
```

```
40     ll n;
41     cin >> n;
42     cout << fib(n) << "\n";
43 }
```

# 2. Ultimos

## 2.1. FordFulkerson

```
1 //O(E*F) donde F flujo maximo
2
3 int dfs(int v, int t, int pushed) {
4     if (v == t) return pushed;
5     vis[v] = 1;
6
7     for (auto &e : G[v]) {
8         if (!vis[e.to] && e.cap > 0) {
9             int f = dfs(e.to, t, min(pushed, e.cap))
10                ;
11             if (f) {
12                 e.cap -= f;
13                 G[e.to][e.rev].cap += f;
14                 return f;
15             }
16         }
17     }
18     return 0;
19 }
20
21 int maxflow(int s, int t) {
22     int flow = 0;
23     while (true) {
24         fill(vis.begin(), vis.end(), 0);
25         int pushed = dfs(s, t, INF);
26         if (!pushed) break;
27         flow += pushed;
28     }
29     return flow;
30 }
```

## 2.2. HoprofKarp

```

1 //complejidad Extraiz(v)
2 struct HopcroftKarp{
3     const int INF = 1e9 + 7;
4     HopcroftKarp(){}
5     HopcroftKarp(int nLeft, int nRight){ init(nLeft,
6         nRight); }
7
8     void init(int nLeft, int nRight){
9         L = nLeft, R = nRight;
10        G.clear(), G.resize(L);
11        pairU.assign(L, -1), pairV.assign(R, -1);
12        dist.resize(L), Q.resize(L);
13    }
14
15    int L, R;
16    vector<vi> G;
17    vi pairU, pairV, dist, Q;
18
19    void addEdge(int u, int v){ G[u].pb(v); }
20
21    bool bfs(){
22        int qh = 0, qt = 0;
23        for(int u = 0; u < L; u++){
24            if(pairU[u] == -1) dist[u] = 0, Q[qt++]
25                = u;
26            else dist[u] = INF;
27        }
28        bool ok = 0;
29        while(qh < qt){
30            int u = Q[qh++];
31            for(int v : G[u]){
32                int u2 = pairV[v];
33                if(u2 == -1) ok = 1;
34                else if(dist[u2] == INF) dist[u2] =
35                    dist[u] + 1, Q[qt++] = u2;
36            }
37        }
38        return ok;
39    }
40
41    bool dfs(int u){
42        for(int v : G[u]){
43            int u2 = pairV[v];

```

```

42            if(u2 == -1 || (dist[u2] == dist[u] + 1
43                && dfs(u2))){
44                pairU[u] = v, pairV[v] = u;
45                return 1;
46            }
47            dist[u] = INF;
48            return 0;
49        }
50
51        int maxMatching(){
52            pairU.assign(L, -1), pairV.assign(R, -1);
53            int match = 0;
54            while(bfs()) for(int u = 0; u < L; u++)
55                if(pairU[u] == -1 && dfs(u)) match++;
56            return match;
57        }
58    };

```

### 2.3. centroid

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const int MAXN=1e5;
6 vector<int> gf[MAXN];
7 bool die[MAXN]={};
8 int ch[MAXN]={};
9 int n,q;
10
11 vector<int> min_dist(MAXN,1e9);
12
13 int get_sz_subTree(int nodo,int p)
14 {
15     ch[nodo]=1;
16     for(int &h:gf[nodo])
17     {
18         if(die[h] or h==p)continue;
19         ch[nodo]+=get_sz_subTree(h,nodo);
20     }
21     return ch[nodo];
22 }

```

```

23
24 int get_centroid(int nodo,int p,int sz)
25 {
26     for(int &h:gf[nodo])
27     {
28         if(die[h] or h==p) continue;
29         if(ch[h]*2>sz) return get_centroid(h,nodo,sz)
30         ;
31     }
32     return nodo;
33 }
34 vector<pair<int,int>> ancenstros[MAXN];
35
36 void get_dist(int nodo,int p,int lvl,int cent)
37 {
38     ancenstros[nodo].push_back({cent,lvl});
39     for(int &h:gf[nodo])
40     {
41         if(h!=p and !die[h])
42         {
43             get_dist(h,nodo,lvl+1,cent);
44         }
45     }
46 }
47
48 void sexo_decomposition(int nodo)
49 {
50     int cent=get_centroid(nodo,-1,get_sz_subTree(
51         nodo,-1));
52     //algo
53     for(int &h:gf[cent])
54     {
55         if(!die[h])
56         {
57             get_dist(h,cent,1,cent);
58         }
59     }
60
61     die[cent]=1;
62     for(int &h:gf[cent]) if(!die[h])
63         sexo_decomposition(h);

```

```

64 }
65
66 void pintar(int nodo)
67 {
68     for(auto &[ancestro,dis]:ancenstros[nodo])
69     {
70         min_dist[ancestro]=min(min_dist[ancestro],
71             dis);
72     }
73     min_dist[nodo]=0;
74 }
75
76 int query(int nodo)
77 {
78     int res=min_dist[nodo];
79
80     for(auto [ancient,dist]:ancenstros[nodo])
81     {
82         res=min(res,dist+min_dist[ancient]);
83     }
84     return res;
85 }
86
87
88 int main()
89 {
90     cin.tie(nullptr)->sync_with_stdio(false);
91     cin>>n>>q;
92
93     for(int i=0;i<n-1;i++)
94     {
95         int a,b;cin>>a>>b;
96         a--;b--;
97         gf[a].push_back(b);
98         gf[b].push_back(a);
99     }
100
101     sexo_decomposition(0);
102
103     pintar(0);
104
105     while(q--)
106     {

```

```

107     int type;cin>>type;
108     int v;cin>>v;v--;
109     if(type&1)
110     {
111         pintar(v);
112     }
113     else
114     {
115         cout<<query(v)<<"\n";
116     }
117 }
118
119
120
121
122 return 0;
123
124 }

```

## 2.4. dinitz

```

1 //O((V^2)*E)
2
3 //Dinic igual complejidad
4
5 struct Dinitz{
6     const int INF = 1e9 + 7;
7     Dinitz(){}
8     Dinitz(int n, int s, int t) {init(n, s, t);}
9
10    void init(int n, int s, int t)
11    {
12        S = s, T = t;
13        nodes = n;
14        G.clear(), G.resize(n);
15        Q.resize(n);
16    }
17    struct flowEdge
18    {
19        int to, rev, f, cap;
20    };
21
22    vector<vector<flowEdge> > G;

```

```

23
24 // Ania de arista (st -> en) con su capacidad
25 void addEdge(int st, int en, int cap) {
26     flowEdge A = {en, (int)G[en].size(), 0, cap};
27     };
28     flowEdge B = {st, (int)G[st].size(), 0, 0};
29     G[st].pb(A);
30     G[en].pb(B);
31 }
32
33 int nodes, S, T; // asignar estos valores al
34 armar el grafo G
35 // nodes = nodos en red de flujo
36 // Hacer G.clear(); G.resize(
37 nodes);
38
39 vi work, lvl;
40 vi Q;
41
42 bool bfs() {
43     int qt = 0;
44     Q[qt++] = S;
45     lvl.assign(nodes, -1);
46     lvl[S] = 0;
47     for (int qh = 0; qh < qt; qh++) {
48         int v = Q[qh];
49         for (flowEdge &e : G[v]) {
50             int u = e.to;
51             if (e.cap <= e.f || lvl[u] != -1)
52                 continue;
53             lvl[u] = lvl[v] + 1;
54             Q[qt++] = u;
55         }
56     }
57     return lvl[T] != -1;
58 }
59
60 int dfs(int v, int f) {
61     if (v == T || f == 0) return f;
62     for (int &i = work[v]; i < G[v].size(); i++)
63     {
64         flowEdge &e = G[v][i];
65         int u = e.to;
66         if (e.cap <= e.f || lvl[u] != lvl[v] +
67             1) continue;

```

```

60         int df = dfs(u, min(f, e.cap - e.f));
61         if (df) {
62             e.f += df;
63             G[u][e.rev].f -= df;
64             return df;
65         }
66     }
67     return 0;
68 }
69
70 int maxFlow() {
71     int flow = 0;
72     while (bfs()) {
73         work.assign(nodes, 0);
74         while (true) {
75             int df = dfs(S, INF);
76             if (df == 0) break;
77             flow += df;
78         }
79     }
80     return flow;
81 }
82 };

```

## 2.5. edmonkarp

```

1 //O(V*(E^2))
2
3 int n;
4 vector<vector<int>> capacity;
5 vector<vector<int>> adj;
6
7 int bfs(int s, int t, vector<int>& parent) {
8     fill(parent.begin(), parent.end(), -1);
9     parent[s] = -2;
10    queue<pair<int, int>> q;
11    q.push({s, INF});
12
13    while (!q.empty()) {
14        int cur = q.front().first;
15        int flow = q.front().second;
16        q.pop();
17

```

```

18        for (int next : adj[cur]) {
19            if (parent[next] == -1 && capacity[cur][
20                next]) {
21                parent[next] = cur;
22                int new_flow = min(flow, capacity[
23                    cur][next]);
24                if (next == t)
25                    return new_flow;
26                q.push({next, new_flow});
27            }
28        }
29    }
30    return 0;
31 }
32
33 int maxflow(int s, int t) {
34     int flow = 0;
35     vector<int> parent(n);
36     int new_flow;
37
38     while (new_flow = bfs(s, t, parent)) {
39         flow += new_flow;
40         int cur = t;
41         while (cur != s) {
42             int prev = parent[cur];
43             capacity[prev][cur] -= new_flow;
44             capacity[cur][prev] += new_flow;
45             cur = prev;
46         }
47     }
48
49     return flow;
50 }

```

## 2.6. extended

```

1 int x, y, d;
2 void extendedEuclid(int a, int b) //ecuacion
3     diofantica ax + by = d
4 {
5     if (b==0) {x=1; y=0; d=a; return;}
6     extendedEuclid(b, a%b);

```

```

6   int x1=y;
7   y = x-(a/b)*y;
8   x=x1;
9 }

```

## 2.7. fastgcd

```

1 int gcd(int a, int b) {
2     if (!a || !b)
3         return a | b;
4     unsigned shift = __builtin_ctz(a | b);
5     a >>= __builtin_ctz(a);
6     do {
7         b >>= __builtin_ctz(b);
8         if (a > b)
9             swap(a, b);
10        b -= a;
11    } while (b);
12    return a << shift;
13 }

```

## 2.8. fft

```

1 using cd = complex<double>;
2 const double PI = acos(-1);
3
4 void fft(vector<cd> & a, bool invert) {
5     int n = a.size();
6
7     for (int i = 1, j = 0; i < n; i++) {
8         int bit = n >> 1;
9         for (; j & bit; bit >>= 1)
10            j ^= bit;
11        j ^= bit;
12
13        if (i < j)
14            swap(a[i], a[j]);
15    }
16
17    for (int len = 2; len <= n; len <<= 1) {
18        double ang = 2 * PI / len * (invert ? -1 :
19            1);

```

```

19        cd wlen(cos(ang), sin(ang));
20        for (int i = 0; i < n; i += len) {
21            cd w(1);
22            for (int j = 0; j < len / 2; j++) {
23                cd u = a[i+j], v = a[i+j+len/2] * w;
24                a[i+j] = u + v;
25                a[i+j+len/2] = u - v;
26                w *= wlen;
27            }
28        }
29    }
30
31    if (invert) {
32        for (cd & x : a)
33            x /= n;
34    }
35 }
36
37 vector<bool> multiply(vector<int> const& a, vector<
38     int> const& b) {
39     vector<cd> fa(a.begin(), a.end()), fb(b.begin(),
40         b.end());
41     int n = 1;
42     while (n < a.size() + b.size())
43         n <<= 1;
44
45     // n=min(n,int(2e6)+10);
46     fa.resize(n);
47     fb.resize(n);
48
49     fft(fa, false);
50     fft(fb, false);
51     for (int i = 0; i < n; i++)
52         fa[i] *= fb[i];
53     fft(fa, true);
54
55     vector<int> result(n);
56     for (int i = 0; i < n; i++)
57         result[i] = round(fa[i].real());
58     return result;
59 }

```

## 2.9. gauss\_algebra\_lineal

```
1 const double EPS = 1e-9;
2 const int INF = 2; // it doesn't actually have to be
   infinity or a big number
3
4
5 int gauss (vector < bitset<N> > a, int n, int m,
   bitset<N> & ans) {
6     vector<int> where (m, -1);
7     for (int col=0, row=0; col<m && row<n; ++col) {
8         for (int i=row; i<n; ++i)
9             if (a[i][col]) {
10                 swap (a[i], a[row]);
11                 break;
12             }
13         if (! a[row][col])
14             continue;
15         where[col] = row;
16
17         for (int i=0; i<n; ++i)
18             if (i != row && a[i][col])
19                 a[i] ^= a[row];
20         ++row;
21     }
22     // The rest of implementation is the same as
       above
23
24     ans.assign (m, 0);
25     for (int i=0; i<m; ++i)
26         if (where[i] != -1)
27             ans[i] = a[where[i]][m] / a[where[i]][i];
28
29     for (int i=0; i<n; ++i) {
30         double sum = 0;
31         for (int j=0; j<m; ++j)
32             sum += ans[j] * a[i][j];
33         if (abs (sum - a[i][m]) > EPS)
34             return 0;
35     }
36
37     for (int i=0; i<m; ++i)
38         if (where[i] == -1)
39             return INF;
```

```
39     return 1;
40 }
```

## 2.10. geometrynotes

```
1 /**
2  *
3  *
4  * Paralelas:          ax * by - ay * bx == 0
5  * Perpendiculares:  ax * bx + ay * by == 0
6  *
7  *
8  * Ecuacion punto-pendiente:
9  * y - y1 = m * (x - x1)
10 *
11 * Pendiente:
12 * m = (y2 - y1) / (x2 - x1)
13 *
14 * Si trabajas con pendientes:
15 * m1 == m2          -> paralelas
16 * m1 * m2 == -1     -> perpendiculares
17 *
18 */
```

## 2.11. hld

```
1
2 const int MAXN=2e5;
3 vector<int> gf[MAXN];
4 int parent[MAXN], depth[MAXN], heavy[MAXN], head[MAXN],
   pos[MAXN];
5 int cur_pos;
6 const ll MOD=1000000007;
7
8 int n;
9
10 int dfs(int nodo)
11 {
12     int size=1;
13     int max_c_size=0;
14     for(int &c:gf[nodo])
15     {
```

```

16         if(c!=parent[nodo])
17         {
18             parent[c]=nodo;
19             depth[c]=depth[nodo]+1;
20
21             int c_size=dfs(c);
22             size+=c_size;
23
24             if(c_size>max_c_size)
25             {
26                 max_c_size=c_size;
27                 heavy[nodo]=c;
28             }
29         }
30     }
31     return size;
32 }
33
34 void update_on_vertex(int a, int b, ll val)
35 {
36     for(; head[a] != head[b]; b = parent[head[b]])
37     {
38         if(depth[head[a]] > depth[head[b]])
39             swap(a,b);
40
41         st.update(1,0,st.sz-1,pos[head[b]],pos[b],
42             val);
43     }
44
45     if(depth[a] > depth[b])
46         swap(a,b);
47
48     st.update(1,0,st.sz-1,pos[a],pos[b],val);
49 }
50 ll query_on_vertex(int a, int b)
51 {
52     ll res = 0;
53
54     for(; head[a] != head[b]; b = parent[head[b]])
55     {
56         if(depth[head[a]] > depth[head[b]])
57             swap(a,b);
58

```

```

59         res = max(res , st.query(1,0,st.sz-1,pos[
60             head[b]],pos[b]));
61     }
62
63     if(depth[a] > depth[b])
64         swap(a,b);
65
66     res = max(res , st.query(1,0,st.sz-1,pos[a],pos[
67         b]));
68     return res;
69 }
70 void decompose(int v,int h)
71 {
72     head[v]=h;
73     pos[v]=cur_pos++;
74     if(heavy[v]!=-1)
75     {
76         decompose(heavy[v],h);
77     }
78     for(int &c:gf[v])
79     {
80         if(c!=parent[v] && c!=heavy[v])
81         {
82             decompose(c,c);
83         }
84     }
85 }
86
87 void init()
88 {
89     // parent=vector<int>(n);
90     // depth=vector<int>(n);
91     // heavy=vector<int>(n);
92     // head=vector<int>(n);
93     // pos=vector<int>(n);
94     for(int i=0;i<MAXN;i++) heavy[i]=-1;
95     cur_pos=0;
96
97     dfs(0);
98     decompose(0,0);
99 }
100

```

```

101 void update(int a,int b, ll val)//es query y update
102     a la vez
103 {
104     for(;head[a]!=head[b];b=parent[head[b]])
105     {
106         if(depth[head[a]]>depth[head[b]]) swap(a,b);
107         // ll resdomer=st.query()
108         st.update(1,0,st.sz-1,pos[head[b]],pos[b],
109             val);
110         // res=(res*resdomer)%MOD;
111     }
112     if(depth[a]>depth[b]) swap(a,b);
113
114     if(a != b)st.update(1,0,st.sz-1,pos[a]+1,pos[b],
115         val);
116 }
117
118 ll query(int a,int b)//quety on edges
119 {
120     ll res=0;
121
122     for(;head[a]!=head[b];b=parent[head[b]])
123     {
124         if(depth[head[a]]>depth[head[b]]) swap(a,b);
125         // ll resdomer=st.query()
126         ll redomer=st.query(1,0,st.sz-1,pos[head[b]
127             ],pos[b]);
128         res=(res+redomer);
129     }
130     if(depth[a]>depth[b]) swap(a,b);
131
132     ll last_heavy_path_res=0;
133     if(a != b)last_heavy_path_res=st.query(1,0,st.sz
134         -1,pos[a]+1,pos[b]);
135
136     // ll last_heavy_path_res=1;
137     res=(res+last_heavy_path_res);
138     return res;
139 }

```

## 2.12. iterativequerysegment

```

1 int query(int l, int r)
2 {
3     int ans = INT_MIN;
4
5     for(l += n, r += n; l <= r; l >>= 1, r >>= 1)
6     {
7         if(l & 1) ans = max(ans, st[l++]);
8         if(!(r & 1)) ans = max(ans, st[r--]);
9     }
10
11     return ans;
12 }

```

## 2.13. khun

```

1 //0(LxE)
2
3 struct Kuhn{
4     Kuhn(){}
5     Kuhn(int nLeft, int nRight){ init(nLeft, nRight)
6         ; }
7
8     void init(int nLeft, int nRight){
9         L = nLeft, R = nRight;
10        G.clear(), G.resize(L);
11        mt.assign(R, -1);
12        used.assign(L, 0);
13        cc = 1;
14    }
15
16    int L, R, cc;
17    vector<vi> G;
18    vi mt, used; // mt[to] = nodo izquierdo
19    emparejado con to
20
21    void addEdge(int u, int v){ G[u].pb(v); }
22
23    bool dfs(int v){
24        if(used[v] == cc) return 0;
25        used[v] = cc;

```

```

24         for(int to : G[v]) if(mt[to] == -1 || dfs(mt
25             [to])) return mt[to] = v, 1;
26         return 0;
27     }
28     int maxMatching(){
29         mt.assign(R, -1);
30         int match = 0;
31         for(int v = 0; v < L; v++){
32             ++cc;
33             if(dfs(v)) match++;
34         }
35         return match;
36     }
37 };

```

## 2.14. latticepoint

```

1 //calculates number of integer points (x ; y) such
  for 0 <= x < n and 0 < y <= piso(k * x + b)
2 int count_lattices(double k, double b, long long n)
  {
3     auto fk = floor(k);
4     auto fb = floor(b);
5     auto cnt = 0LL;
6     if (k > 1 - EPS || b > 1 - EPS) {
7         cnt += (fk * (n - 1) + 2 * fb) * n / 2;
8         k -= fk;
9         b -= fb;
10    }
11    auto t = k * n + b;
12    auto ft = floor(t);
13    if (ft > 1 - EPS) {
14        cnt += count_lattices(1 / k, (t - floor(t))
15            / k, floor(t));
16    }
17    return cnt;
18 }
19 pair<double, double> kb(point a, point b)
20 {
21     return mp((b.y - a.y) / (b.x - a.x), (a.y * b.x -
    a.x * b.y) / (b.x - a.x));
22 }

```

```

22 // # of lattice points s.t. ax+by<=c, x,y>0 (a,b is
23     positive integer)
24 ll f(ll a, ll b, ll c){
25     if(c<=0) return 0;
26     if(a<b) swap(a, b);
27     ll m=c/a;
28     if(a==b) return m*(m-1)/2;
29     ll k=(a-1)/b, h=(c-a*m)/b;
30     return f(b,a-b*k,c-b*(k*m+h))+k*m*(m-1)/2+m*h;
31 }
32
33 // # of lattice points s.t. ax+by<=c, 0<x<=X, 0<y<=Y
34     (a,b is positive integer)
35 ll g(ll a, ll b, ll c, ll X, ll Y){
36     if(a*X+b*Y<=c) return X*Y;
37     return f(a,b,c)-f(a,b,c-a*X)-f(a,b,c-b*Y)+f(a,b,c-
    a*X-b*Y);
38 }

```

## 2.15. lcs

```

1 int longestCommonSubsequence(string text1, string
  text2) {
2     int n = text1.size(), m = text2.size();
3     vector<vector<int>> lcs(n+1, vector<int>(m+1));
4     for(int i = 1; i <= n; i++)
5         for(int j = 1; j <= m; j++)
6         {
7             lcs[i][j] = max(lcs[i - 1][j], lcs[i][j - 1]);
8             if(text1[i - 1] == text2[j - 1])
9                 lcs[i][j] = max(lcs[i][j], lcs[i - 1][j - 1] +
10                     1);
11         }
12     return lcs[n][m];
13 }

```

## 2.16. linearsieve

```

1 std::vector <int> prime;
2 bool is_composite[MAXN];
3

```

```

4 void sieve (int n) {
5     std::fill (is_composite, is_composite + n, false);
6     for (int i = 2; i < n; ++i) {
7         if (!is_composite[i]) prime.push_back (i);
8         for (int j = 0; j < prime.size () && i * prime[j]
9             ] < n; ++j) {
10             is_composite[i * prime[j]] = true;
11             if (i % prime[j] == 0) break;
12         }
13     }
14
15     // As is shown in the code, the statement if (i %
16     // prime[j] == 0) break;
17     // terminates the loop when p divides i.
18     // From the analysis above, we can see that the
19     // inner loop is executed only once for each
20     // composite.
21     // Hence, the code above performs in O(n) complexity
22     // , resulting in its name - the 'linear' sieve.

```

## 2.17. lis

```

1 // Longest Increasing Subsequence - O(nlogn)
2 //
3 // dp(i) = max j<i { dp(j) | a[j] < a[i] } + 1
4 //
5
6 // int dp[N], v[N], n, lis;
7
8 memset(dp, 63, sizeof dp);
9 for (int i = 0; i < n; ++i) {
10     // increasing: lower_bound
11     // non-decreasing: upper_bound
12     int j = lower_bound(dp, dp + lis, v[i]) - dp;
13     dp[j] = min(dp[j], v[i]);
14     lis = max(lis, j + 1);
15 }

```

## 2.18. nearestVecino

```

1 // Closest Neighbor - O(n * log(n))

```

```

2 const ll N = 1e6+3, INF = 1e18;
3 ll n, cn[N], x[N], y[N]; // number of points, closes
4                             neighbor, x coordinates, y coordinates
5
6 ll sqr(ll i) { return i*i; }
7 ll dist(int i, int j) { return sqr(x[i]-x[j]) + sqr(
8     y[i]-y[j]); }
9 ll dist(int i) { return i == cn[i] ? INF : dist(i,
10     cn[i]); }
11
12 bool cpx(int i, int j) { return x[i] < x[j] or (x[i]
13     == x[j] and y[i] < y[j]); }
14 bool cpy(int i, int j) { return y[i] < y[j] or (y[i]
15     == y[j] and x[i] < x[j]); }
16
17 ll calc(int i, ll x0) {
18     ll dlt = dist(i) - sqr(x[i]-x0);
19     return dlt >= 0 ? ceil(sqrt(dlt)) : -1;
20 }
21
22 void updt(int i, int j, ll x0, ll &dlt) {
23     if (dist(i) > dist(i, j)) cn[i] = j, dlt = calc(i,
24         x0);
25 }
26
27 void cmp(vi &u, vi &v, ll x0) {
28     for(int a=0, b=0; a<u.size(); ++a) {
29         ll i = u[a], dlt = calc(i, x0);
30         while(b < v.size() and y[i] > y[v[b]]) b++;
31         for(int j = b-1; j >= 0 and y[i] - dlt <= y[
32             v[j]]; j--) updt(i, v[j], x0, dlt);
33         for(int j = b; j < v.size() and y[i] + dlt >= y[
34             v[j]]; j++) updt(i, v[j], x0, dlt);
35     }
36 }
37
38 void slv(vi &ix, vi &iy) {
39     int n = ix.size();
40     if (n == 1) { cn[ix[0]] = ix[0]; return; }
41
42     int m = ix[n/2];
43
44     vi ix1, ix2, iy1, iy2;
45     for(int i=0; i<n; ++i) {

```

```

38     if (cpx(ix[i], m)) ix1.push_back(ix[i]);
39     else ix2.push_back(ix[i]);
40
41     if (cpx(iy[i], m)) iy1.push_back(iy[i]);
42     else iy2.push_back(iy[i]);
43 }
44
45 slv(ix1, iy1);
46 slv(ix2, iy2);
47
48 cmp(iy1, iy2, x[m]);
49 cmp(iy2, iy1, x[m]);
50 }
51
52 void slv(int n) {
53     vi ix, iy;
54     ix.resize(n);
55     iy.resize(n);
56     for(int i=0; i<n; ++i) ix[i] = iy[i] = i;
57     sort(ix.begin(), ix.end(), cpx);
58     sort(iy.begin(), iy.end(), cpy);
59     slv(ix, iy);
60 }

```

## 2.19. pointintpoligon

```

1 struct pt {
2     long long x, y;
3     pt() {}
4     pt(long long _x, long long _y) : x(_x), y(_y) {}
5     pt operator+(const pt &p) const { return pt(x +
6         p.x, y + p.y); }
7     pt operator-(const pt &p) const { return pt(x -
8         p.x, y - p.y); }
9     long long cross(const pt &p) const { return x *
10        p.y - y * p.x; }
11    long long dot(const pt &p) const { return x * p.
12        x + y * p.y; }
13    long long cross(const pt &a, const pt &b) const
14        { return (a - *this).cross(b - *this); }
15    long long dot(const pt &a, const pt &b) const {
16        return (a - *this).dot(b - *this); }
17    long long sqrLen() const { return this->dot(*

```

```

18        this); }
19 };
20
21 bool lexComp(const pt &l, const pt &r) {
22     return l.x < r.x || (l.x == r.x && l.y < r.y);
23 }
24
25 int sgn(long long val) { return val > 0 ? 1 : (val
26     == 0 ? 0 : -1); }
27
28 vector<pt> seq;
29 pt translation;
30 int n;
31
32 bool pointInTriangle(pt a, pt b, pt c, pt point) {
33     long long s1 = abs(a.cross(b, c));
34     long long s2 = abs(point.cross(a, b)) + abs(
35         point.cross(b, c)) + abs(point.cross(c, a));
36     return s1 == s2;
37 }
38
39 void prepare(vector<pt> &points) {
40     n = points.size();
41     int pos = 0;
42     for (int i = 1; i < n; i++) {
43         if (lexComp(points[i], points[pos]))
44             pos = i;
45     }
46     rotate(points.begin(), points.begin() + pos,
47         points.end());
48
49     n--;
50     seq.resize(n);
51     for (int i = 0; i < n; i++)
52         seq[i] = points[i + 1] - points[0];
53     translation = points[0];
54 }
55
56 bool pointInConvexPolygon(pt point) {
57     point = point - translation;
58     if (seq[0].cross(point) != 0 &&
59         sgn(seq[0].cross(point)) != sgn(seq[0].
60             cross(seq[n - 1])))
61         return false;

```

```

51     if (seq[n - 1].cross(point) != 0 &&
52         sgn(seq[n - 1].cross(point)) != sgn(seq[
53             n - 1].cross(seq[0])))
54         return false;
55
56     if (seq[0].cross(point) == 0)
57         return seq[0].sqrLen() >= point.sqrLen();
58
59     int l = 0, r = n - 1;
60     while (r - l > 1) {
61         int mid = (l + r) / 2;
62         int pos = mid;
63         if (seq[pos].cross(point) >= 0)
64             l = mid;
65         else
66             r = mid;
67     }
68     int pos = l;
69     return pointInTriangle(seq[pos], seq[pos + 1],
70         pt(0, 0), point);
71 }

```

## 2.20. starsandbars

```

1  /**
2   * (n+k-1,n)
3   *
4   * Si cada caja tiene que tener al menos uno
5   * (n-1,k-1)
6   *
7   *
8   */

```

## 2.21. suffixarray

```

1  vector<int> suffixa(string &s){
2      int n = s.size(), cc, ax;
3      vector<int> sa(n), sa1(n), col(n), col1(n), head(n)
4          );
5      fore(i, 0, n) sa[i] = i;
6      auto cmp = [&](int a, int b){ return s[a] < s[b];
7          };

```

```

6      stable_sort(sa.begin(), sa.end(), cmp);
7      head[0] = col[sa[0]] = cc = 0;
8      fore(i, 1, n){
9          if(s[sa[i]] != s[sa[i-1]])
10             cc++, head[cc] = i;
11             col[sa[i]] = cc;
12     }
13     for(int k = 1; k < n; k *= 2){
14         fore(i, 0, n){
15             ax = (sa[i] - k + n) % n;
16             sa1[head[col[ax]]++] = ax;
17         }
18         swap(sa, sa1);
19         col1[sa[0]] = head[0] = cc = 0;
20         fore(i, 1, n){
21             if(col[sa[i]] != col[sa[i - 1]] || col[(sa[i]
22                 + k) % n] != col[(sa[i - 1] + k) % n])
23                 cc++, head[cc] = i;
24                 col1[sa[i]] = cc;
25         }
26         swap(col, col1);
27         if(col[sa[n - 1]] == n - 1) break;
28     }
29     return sa;
30 }
31 vector<int> lcp(string &s, vector<int> &sa){
32     int n = s.size(), k, z = 0;
33     vector<int> sa1(n), lcp(n);
34     fore(i, 0, n) sa1[sa[i]] = i;
35     fore(i, 0, n){
36         k = sa1[i];
37         if(k < n - 1)
38             while(s[i + z] == s[sa[k+1] + z])
39                 z++;
40         lcp[k] = z; z = max(z-1, 0);
41     }
42     return lcp;

```

## 2.22. trie

```

1  #include <bits/stdc++.h>
2  using namespace std;

```

```

3
4 struct Trie {
5     static constexpr int ALPHABET = 26;
6
7     struct Node {
8         int next[ALPHABET];
9         int prefixCount;
10        int wordCount;
11
12        Node() {
13            fill(next, next + ALPHABET, -1);
14            prefixCount = 0;
15            wordCount = 0;
16        }
17    };
18
19    vector<Node> trie;
20
21    Trie(int maxNodes = 1) {
22        trie.reserve(maxNodes);
23        trie.emplace_back(); // Nodo 0: raiz
24    }
25
26    // Reinicia completamente el Trie
27    void clear() {
28        trie.clear();
29        trie.emplace_back();
30    }
31
32    // Crea un nuevo nodo y devuelve su indice
33    int newNode() {
34        trie.emplace_back();
35        return (int)trie.size() - 1;
36    }
37
38    // Inserta una palabra
39    // Supone que contiene solamente letras entre 'a
40    ' y 'z'
41    void insert(const string& word) {
42        int node = 0;
43        trie[node].prefixCount++;
44
45        for (char ch : word) {
46            int c = ch - 'a';

```

```

46
47        if (trie[node].next[c] == -1) {
48            trie[node].next[c] = newNode();
49        }
50
51        node = trie[node].next[c];
52        trie[node].prefixCount++;
53    }
54
55    trie[node].wordCount++;
56 }
57
58 // Devuelve el nodo donde termina la cadena
59 // Devuelve -1 si la cadena no existe como
60 // camino en el Trie
61 int findNode(const string& word) const {
62     int node = 0;
63
64     for (char ch : word) {
65         int c = ch - 'a';
66
67         if (c < 0 || c >= ALPHABET)
68             return -1;
69
70         if (trie[node].next[c] == -1)
71             return -1;
72
73         node = trie[node].next[c];
74     }
75
76     return node;
77 }
78
79 // Devuelve cuantas veces se inserto exactamente
80 // una palabra
81 int countWord(const string& word) const {
82     int node = findNode(word);
83
84     if (node == -1)
85         return 0;
86
87     return trie[node].wordCount;
88 }

```

```

88 // Indica si una palabra fue insertada al menos
    una vez
89 bool contains(const string& word) const {
90     return countWord(word) > 0;
91 }
92
93 // Devuelve cuantas palabras insertadas
    comienzan con prefix
94 int countPrefix(const string& prefix) const {
95     int node = findNode(prefix);
96
97     if (node == -1)
98         return 0;
99
100     return trie[node].prefixCount;
101 }
102
103 // Elimina una aparicion de la palabra
104 // Devuelve false si la palabra no estaba
    insertada
105 bool erase(const string& word) {
106     if (!contains(word))
107         return false;
108
109     int node = 0;
110     trie[node].prefixCount--;
111
112     for (char ch : word) {
113         int c = ch - 'a';
114         node = trie[node].next[c];
115         trie[node].prefixCount--;
116     }
117
118     trie[node].wordCount--;
119     return true;
120 }
121
122 // Devuelve la cantidad de nodos utilizados
123 int size() const {
124     return (int)trie.size();
125 }
126 };

```

## 2.23. trie2

```

1 const int K = 26;
2
3 struct Vertex {
4     int next[K];
5     bool output = false;
6
7     Vertex() {
8         fill(begin(next), end(next), -1);
9     }
10 };
11
12 vector<Vertex> trie(1);
13
14 void add_string(string const& s) {
15     int v = 0;
16     for (char ch : s) {
17         int c = ch - 'a';
18         if (trie[v].next[c] == -1) {
19             trie[v].next[c] = trie.size();
20             trie.emplace_back();
21         }
22         v = trie[v].next[c];
23     }
24     trie[v].output = true;
25 }

```

## 2.24. twoSegmentIntersec

```

1 struct pt {
2     long long x, y;
3     pt() {}
4     pt(long long _x, long long _y) : x(_x), y(_y) {}
5     pt operator-(const pt& p) const { return pt(x -
        p.x, y - p.y); }
6     long long cross(const pt& p) const { return x *
        p.y - y * p.x; }
7     long long cross(const pt& a, const pt& b) const
        { return (a - *this).cross(b - *this); }
8 };
9

```

```

10 int sgn(const long long& x) { return x >= 0 ? x : 1
    : 0 : -1; }
11
12 bool inter1(long long a, long long b, long long c,
    long long d) {
13     if (a > b)
14         swap(a, b);
15     if (c > d)
16         swap(c, d);
17     return max(a, c) <= min(b, d);
18 }
19
20 bool check_inter(const pt& a, const pt& b, const pt&
    c, const pt& d) {
21     if (c.cross(a, d) == 0 && c.cross(b, d) == 0)
22         return inter1(a.x, b.x, c.x, d.x) && inter1(
            a.y, b.y, c.y, d.y);
23     return sgn(a.cross(b, c)) != sgn(a.cross(b, d))
        &&
24         sgn(c.cross(d, a)) != sgn(c.cross(d, b));
25 }

```

### 3. Updates

#### 3.1. 2-sat

```

1 struct TwoSatSolver {
2     int n_vars;
3     int n_vertices;
4     vector<vector<int>> adj, adj_t;
5     vector<bool> used;
6     vector<int> order, comp;
7     vector<bool> assignment;
8
9     TwoSatSolver(int n_vars) : n_vars(n_vars),
        n_vertices(2 * n_vars), adj(n_vertices),
        adj_t(n_vertices), used(n_vertices), order(),
        comp(n_vertices, -1), assignment(n_vars) {
10         order.reserve(n_vertices);
11     }
12     void dfs1(int v) {
13         used[v] = true;
14         for (int u : adj[v]) {

```

```

15             if (!used[u])
16                 dfs1(u);
17         }
18         order.push_back(v);
19     }
20
21     void dfs2(int v, int cl) {
22         comp[v] = cl;
23         for (int u : adj_t[v]) {
24             if (comp[u] == -1)
25                 dfs2(u, cl);
26         }
27     }
28
29     bool solve_2SAT() {
30         order.clear();
31         used.assign(n_vertices, false);
32         for (int i = 0; i < n_vertices; ++i) {
33             if (!used[i])
34                 dfs1(i);
35         }
36
37         comp.assign(n_vertices, -1);
38         for (int i = 0, j = 0; i < n_vertices; ++i)
39             {
40                 int v = order[n_vertices - i - 1];
41                 if (comp[v] == -1)
42                     dfs2(v, j++);
43             }
44
45         assignment.assign(n_vars, false);
46         for (int i = 0; i < n_vertices; i += 2) {
47             if (comp[i] == comp[i + 1])
48                 return false;
49             assignment[i / 2] = comp[i] > comp[i +
                1];
50         }
51         return true;
52     }
53
54     void add_disjunction(int a, bool na, int b, bool
        nb) {
55         // na and nb signify whether a and b are to
56         // be negated

```

```

55     a = 2 * a ^ na;
56     b = 2 * b ^ nb;
57     int neg_a = a ^ 1;
58     int neg_b = b ^ 1;
59     adj[neg_a].push_back(b);
60     adj[neg_b].push_back(a);
61     adj_t[b].push_back(neg_a);
62     adj_t[a].push_back(neg_b);
63 }
64
65 static void example_usage() {
66     TwoSatSolver solver(3); // a, b, c
67     solver.add_disjunction(0, false, 1, true);
68     // a v not b
69     solver.add_disjunction(0, true, 1, true);
70     // not a v not b
71     solver.add_disjunction(1, false, 2, false);
72     // b v c
73     solver.add_disjunction(0, false, 0, false);
74     // a v a
75     assert(solver.solve_2SAT() == true);
76     auto expected = vector<bool>(True, False,
77     True);
78     assert(solver.assignment == expected);
79 }
80 };

```

### 3.2. ahocorasick

```

1  const int K = 26;
2
3  struct Vertex {
4      int next[K];
5      bool output = false;
6      int p = -1;
7      char pch;
8      int link = -1;
9      int go[K];
10
11     Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
12         fill(begin(next), end(next), -1);
13         fill(begin(go), end(go), -1);
14     }

```

```

15 };
16
17 vector<Vertex> t(1);
18
19 void add_string(string const& s) {
20     int v = 0;
21     for (char ch : s) {
22         int c = ch - 'a';
23         if (t[v].next[c] == -1) {
24             t[v].next[c] = t.size();
25             t.emplace_back(v, ch);
26         }
27         v = t[v].next[c];
28     }
29     t[v].output = true;
30 }
31
32 int go(int v, char ch);
33
34 int get_link(int v) {
35     if (t[v].link == -1) {
36         if (v == 0 || t[v].p == 0)
37             t[v].link = 0;
38         else
39             t[v].link = go(get_link(t[v].p), t[v].
40             pch);
41     }
42     return t[v].link;
43 }
44
45 int go(int v, char ch) {
46     int c = ch - 'a';
47     if (t[v].go[c] == -1) {
48         if (t[v].next[c] != -1)
49             t[v].go[c] = t[v].next[c];
50         else
51             t[v].go[c] = v == 0 ? 0 : go(get_link(v),
52             ch);
53     }
54     return t[v].go[c];
55 }

```

### 3.3. componentes\_fuertemente\_conexos

```
1 #include <bits/stdc++.h>
2
3 const int N = 1e5 + 1;
4
5 // adj_t is the transpose of adj
6 std::vector<int> adj[N], adj_t[N];
7 std::vector<int> order;
8 std::vector<int> vis(N), id(N);
9
10 // calculates the order in which nodes are processed
11 void dfs1(int v) {
12     vis[v] = true;
13     for (int u : adj[v]) {
14         if (!vis[u]) { dfs1(u); }
15     }
16     order.push_back(v);
17 }
18
19 /*
20 * traverse one SCC at a time and
21 * assign component ID to all nodes in one SCC
22 */
23 void dfs2(int x, int comp) {
24     vis[x] = true;
25
26     for (int u : adj_t[x]) {
27         if (!vis[u]) { dfs2(u, comp); }
28     }
29     id[x] = comp;
30 }
31
32 int main() {
33     int n, m;
34     std::cin >> n >> m;
35
36     for (int i = 1; i <= m; i++) {
37         int u, v;
38         std::cin >> u >> v;
39         adj[u].push_back(v);
40         adj_t[v].push_back(u);
41     }
42 }
```

```
43 vis.assign(N, false);
44
45 for (int i = 1; i <= n; i++) {
46     if (!vis[i]) { dfs1(i); }
47 }
48
49 vis.assign(N, false);
50 std::reverse(begin(order), end(order));
51 int comps = 0;
52 for (int v : order) {
53     if (!vis[v]) {
54         comps++; // counter for SCC
55         dfs2(v, comps);
56     }
57 }
58
59 std::cout << comps << '\n';
60
61 for (int i = 1; i <= n; i++) { std::cout << id[i]
62     << " \n"[i == n]; }
63 }
```

### 3.4. floyd\_warshall

```
1 for (int k = 0; k < n; ++k) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             if (d[i][k] < INF && d[k][j] < INF)
5                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
6         }
7     }
8 }
```

### 3.5. lazy

```
1 struct SegmentTreeLazy
2 {
3     void push(int nodo, int l_n, int r_n){
4
5         /**
6          * 1. Aplicar lazy del padre(nodo) a los
           hijos
```

```

7         * 2.marcar el lazy a los hijos
8         * 3.matar el lazy del padre */
9
10    }
11
12    long long query(int nodo, int l_n,int r_n,int
13        l_q,int r_q){
14        if(l_q<=l_n and r_n<=r_q) return st[nodo].sc
15        ;
16        if(l_q>r_n or l_n>r_q) return 0;
17        push(nodo,l_n,r_n);
18        int m=(l_n+r_n)>>1;
19        return (long long)((query(2*nodo,l_n,m,l_q,
20            r_q)+query(2*nodo+1,m+1,r_n,l_q,r_q)));
21    }
22    void update(int nodo, int l_n,int r_n,int l_q,
23        int r_q,ll val){
24        if(l_q>r_n or l_n>r_q) return;
25        if(l_q<=l_n and r_n<=r_q){
26            //aplicar el lazy, osea si fuera suma
27            sumarlo al nodo y guardar el lazy
28            st[nodo].lazy_x+=val;
29            return;
30        }
31        else{
32            push(nodo,l_n,r_n);
33            int m=(l_n+r_n)/2;
34            update(2*nodo,l_n,m,l_q,r_q,val);
35            update(2*nodo+1,m+1,r_n,l_q,r_q,val);
36            st[nodo]=st[nodo*2]+st[nodo*2+1];
37        }
38    }
39 };

```

## 4. Bits

### 4.1. SoSDP

```

1 //suma de subsets,donde A[i]tiene la suma de esa
2 mascara
3 for(int i = 0; i<(1<<N); ++i)ans[i] = A[i];
4 for(int i = 0;i < N; ++i){
5     for(int mask = 0; mask < (1<<N); ++mask){

```

```

5         if(mask & (1<<i))ans[mask] += ans[mask^(1<<i
6             )]];
7     }
8 }

```

## 5. Miscelanea

### 5.1. Plantilla

```

1 #include <bits/stdc++.h>
2 #include <bits/extc++.h>
3 using namespace std;
4 using namespace __gnu_pbds;
5 typedef long long ll;
6 typedef long double ld;
7
8 typedef tree<pair<ll, ll>, null_type, less<pair<ll,
9     ll>>, rb_tree_tag,
10     tree_order_statistics_node_update>
11     ordered_set_men;
12 typedef tree<int, null_type, greater<int>,
13     rb_tree_tag, tree_order_statistics_node_update>
14     ordered_set_may;
15
16 #define CRISTIANO RONALDO GANO 35 COPAS \
17 ios_base::sync_with_stdio(false); \
18 cin.tie(NULL); \
19 cout.tie(nullptr);
20
21 #define hola cout << "hola" << endl;
22 #define YES cout << "YES" << endl;
23 #define NO cout << "NO" << endl;
24 #define dbg(x) cout << #x << ": " << x << endl;
25 #define dbg2(x, y) cout << #x << ": " << x << " __ "
26 << #y << ": " << y << endl;
27
28 #define printvii(v_v) \
29     for (auto [x_x, y_y] : v_v) \
30     { \
31         cout << x_x << " " << y_y << endl; \
32     } \
33     cout << endl;
34
35 #define printst(st) \
36     for (auto num : st) \

```

```

29     cout << num << " "; \
30     cout << endl;
31 template <typename T>
32 void printv(T v)
33 {
34     for (auto x : v)
35         cout << x << " ";
36     cout << endl;
37 }
38
39 #define RAYA cout << "-----"
40     << endl;
41 // #define F first
42 // #define S second
43 const ll MOD = 1'000'000'007;
44 const vector<int> F = {0, 1, 0, -1};
45 const vector<int> C = {1, 0, -1, 0};
46
47 int main()
48 {
49     CRISTIANO RONALDO GANO 35 COPAS
50     //sol
51     return 0;

```

## 6. Datastructures

### 6.1. LowestCommonAncestor

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int n, m; //nodos, links
6 vector<int> h; //altura del nodo
7 vector<int> primera; //primera aparicion
8 // set<int> st;
9 vector<bool> vis;
10 vector<vector<int>> gf;
11 vector<int> nodo; //nodo
12
13 void dfs(int currt, int alt)
14 {

```

```

15     if (!vis[currt])
16     {
17         primera[currt] = h.size();
18     }
19     vis[currt] = 1;
20     h.push_back(alt);
21     nodo.push_back(currt);
22
23     for (int hijo : gf[currt])
24     {
25         if (!vis[hijo])
26         {
27             dfs(hijo, alt + 1);
28             h.push_back(alt);
29             nodo.push_back(currt);
30         }
31     }
32 }
33
34 int main()
35 {
36     vis.assign(n, 0);
37     primera.assign(n, -1);
38     gf.assign(n, vector<int>());
39     // st.clear();
40     h.clear();
41     nodo.clear();
42     //0i
43     dfs(0, 0);
44
45     //hacer un segmet tree sobre h que es la altura,
46     //y responder cout<<nodo[res] donde res es el
47     //indice del minimo en el rango, para minimo en
48     //rango, donde l y r son los dos nodos
49     return 0;

```

### 6.2. Mo

```

1 struct Query
2 {
3     int l, r, idx;
4     Query() {}

```

```

5     Query(int lx, int rr, int ii)
6     {
7         l = lx;
8         r = rr;
9         idx = ii;
10    }
11};
12
13
14
15inline void add(int i)
16{
17    //hacer algo
18}
19
20inline void remove(int i)
21{
22    //hacer algo
23}
24
25inline bool get_answer()
26{
27    return imp == 0;
28}
29
30inline vector<bool> Mo(vector<Query> &qs)
31{
32    const int szb = int(ceil(sqrt(N + .0)));
33    sort(qs.begin(), qs.end(), [&](const Query &a,
34        const Query &b)
35    {
36        if (a.l / szb == b.l / szb)
37        {
38            return ((a.l / szb) & 1) ? a.r < b.r : a
39                .r > b.r;
40        }
41        return a.l < b.l;
42    });
43    vector<bool> answers(Q);
44    int c_l = 0, c_r = -1;
45    for (auto &q : qs)
46    {
47        int ql = q.l;
48        int qr = q.r;

```

```

47        int qi = q.idx;
48        while (c_l > ql)
49        {
50            c_l--;
51            add(c_l);
52        }
53        while (c_r < qr)
54        {
55            c_r++;
56            add(c_r);
57        }
58        while (c_l < ql)
59        {
60            remove(c_l);
61            c_l++;
62        }
63        while (c_r > qr)
64        {
65            remove(c_r);
66            c_r--;
67        }
68        answers[qi] = get_answer();
69    }
70    return answers;
71}
72
73int main()
74{
75    CRISTIANO RONALDO GANO 35 COPAS;
76
77    int t;
78    cin >> t;
79    while (t--)
80    {
81        cin >> N >> Q;
82
83        vec.assign(N, 0);
84
85        for (int i = 0; i < N; i++)
86            cin >> vec[i];
87
88        vector<Query> qs(Q);
89        for (int i = 0; i < Q; i++)
90        {

```

```

91         int l, r;
92         cin >> l >> r;
93         l--;
94         r--;
95         qs[i] = Query(l, r, i);
96     }
97
98     ans = 0;
99     imp = 0;
100    par = 0;
101    vector<bool> ans = Mo(qs);
102
103    for (int i = 0; i < Q; i++)
104    {
105        if (ans[i])
106            cout << "YES\n";
107        else
108            cout << "NO\n";
109    }
110    // dbg(mp.size());
111    for(int i=0;i<N;i++)mp[vec[i]]=0;
112 }
113
114 return 0;
115 }

```

### 6.3. QueueMin

```

1 struct quemini
2 {
3     stack<pair<int,int>> bo, to;
4     void push(int n)
5     {
6         if(bo.empty())
7             bo.push(mp(n, n));
8         else
9             bo.push(mp(n, min(bo.top().s, n)));
10    }
11    void pop()
12    {
13        if(to.empty())
14        {
15            while(!bo.empty())

```

```

16        {
17            if(to.empty())
18                to.push(mp(bo.top().f, bo.top().f));
19            else
20                to.push(mp(bo.top().f, min(bo.top().f, to.
21                    top().s)));
22            bo.pop();
23        }
24    }
25    to.pop();
26 }
27 int mini()
28 {
29     int mini = MOD;
30     if(!bo.empty())
31         mini = bo.top().s;
32     if(!to.empty())
33         mini = min(mini, to.top().s);
34     return mini;
35 }
36 };
37 struct quemini
38 {
39     pair<int,int> bo[100010], to[100010];
40     int boto = -1, toto = -1, ax;
41     void push(int n)
42     {
43         ax = boto + 1;
44         if(boto == -1)
45             bo[ax] = mp(n, n);
46         else
47             bo[ax] = mp(n, min(bo[boto].s, n));
48         boto++;
49     }
50     void pop()
51     {
52         if(toto == -1)
53         {
54             while(boto > -1)
55             {
56                 ax = toto + 1;
57                 if(toto == -1)
58                     to[ax] = mp(bo[boto].f, bo[boto].f);

```

```

59         else
60             to[ax] = mp(bo[boto].f, min(bo[boto].f, to
61                                     [toto].s));
62             toto++;
63             boto--;
64         }
65         if(toto > -1)
66             toto--;
67     }
68     int mini()
69     {
70         int mini = MOD;
71         if(boto > -1)
72             mini = bo[boto].s;
73         if(toto > -1)
74             mini = min(mini, to[toto].s);
75         return mini;
76     }
77 };

```

#### 6.4. SegmentTree

```

1 struct Node
2 {
3     //constructor
4     Node operator+(const Node &other) const
5     {
6         //merge
7     }
8 };
9
10 struct SegmentTree
11 {
12     int sz;
13     vector<Node> st;
14
15     SegmentTree(int n, vector<ll> &v)
16     {
17         if(__builtin_popcount(n) != 1) n = (1 << (31 -
18             __builtin_clz(n) + 1));
19         sz = n;
20         st.assign(2 * sz, Node());

```

```

20         for(int i=0; i<(int)v.size(); i++)
21         {
22             st[sz+i] = Node(v[i]);
23         }
24         for(int i=2*sz-1; i>=2; i-=2)
25         {
26             int p=i/2;
27             st[p]=st[p*2]+st[p*2+1]; //orden
28                                     importante aaaaa media hora
29                                     debuguenado mierda aaaaaa sex0000
30         }
31     }
32
33     void update(ll num, int pos) //0i
34     {
35         st[sz+pos] = Node(num);
36         pos += sz;
37         while(pos > 1)
38         {
39             int par=pos/2;
40             st[par]=st[2*par]+st[2*par+1];
41             pos=par;
42         }
43     }
44
45     Node query(int nodo, int l_n, int r_n, int l_q, int
46               r_q)
47     {
48         //si tengo todo retorna todo
49         if(l_q <= l_n and r_n <= r_q) return st[nodo];
50         // si no tengo nada retorna neutro
51         if(l_q > r_n or l_n > r_q) return Node(); //neutro
52         int m=(l_n+r_n)/2;
53         return query(2*nodo, l_n, m, l_q, r_q) + query(2*
54               nodo+1, m+1, r_n, l_q, r_q);
55     }
56
57     //(1,0,sz-1,l_q,r_q,val)[0i]s
58     int get_first(int v, int tl, int tr, int l, int
59               r, int x)
60     {
61         if(tl > r || tr < l) return -1;
62         if(t[v] <= x) return -1;

```

```

59         if (t1== tr) return t1;
60
61         int tm = t1 + (tr-t1)/2;
62         int left = get_first(2*v, t1, tm, l, r, x);
63         if(left != -1) return left;
64         return get_first(2*v+1, tm+1, tr, l ,r, x);
65     }
66 };

```

## 6.5. SparseTable

```

1 struct SparseTable {
2     int n;
3     vector<int> log2;
4     vector<vector<int>> st;
5
6     SparseTable(const vector<int>& a) {
7         n = a.size();
8         log2.resize(n + 1);
9         log2[1] = 0;
10        for (int i = 2; i <= n; i++)
11            log2[i] = log2[i/2] + 1;
12
13        int k = log2[n] + 1;
14        st.assign(n, vector<int>(k));
15
16        for (int i = 0; i < n; i++) st[i][0] = a[i];
17
18        for (int j = 1; j < k; j++)
19            for (int i = 0; i + (1 << j) <= n; i++)
20                st[i][j] = min(st[i][j-1], st[i + (1
21                    << (j-1))][j-1]);
22
23        // Consulta de minimo en el rango [l, r]oi
24        int query(int l, int r) {
25            int j = log2[r - l + 1];
26            return min(st[l][j], st[r - (1 << j) + 1][j
27                ]);
28        }
29 };

```

## 6.6. UnionFind

```

1 struct unionFind {
2     vi p;
3     unionFind(int n) : p(n, -1) {}
4     int findParent(int v) {
5         if (p[v] == -1) return v;
6         return p[v] = findParent(p[v]);
7     }
8     bool join(int a, int b) {
9         a = findParent(a);
10        b = findParent(b);
11        if (a == b) return false;
12        p[a] = b;
13        return true;
14    }
15 };

```

## 6.7. UnionFind2D

```

1 const int MAX=1000000+10;
2 vector<int> sz(MAX,1);
3 vector<int> par(MAX,-1);
4 int n,m;
5
6 int comp=0;
7
8
9 int id(int x,int y)
10 {
11     return x*m+y;
12 }
13
14 int find(int v)
15 {
16     if(v==par[v]) return v;
17     return par[v]=find(par[v]);
18 }
19
20 int find(int x,int y)
21 {
22     return find(id(x,y));
23 }

```

```

24
25 void unite(int x1,int y1,int x2,int y2)
26 {
27     int a=find(x1,y1);
28     int b=find(x2,y2);
29
30     if(a!=b)
31     {
32         if(sz[a]<sz[b]) swap(a,b);
33
34         par[b]=a;
35         sz[a]+=sz[b];
36     }
37 }
38
39 bool same(int x1,int y1,int x2,int y2)
40 {
41     return find(x1,y1)==find(x2,y2);
42 }
43
44 int size(int x,int y)
45 {
46     return sz[find(x,y)];
47 }

```

## 6.8. fenwickTree

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4 typedef long long ll;
5
6
7 struct Bit
8 {
9     private:
10         vector<ll> bit;
11         int sz;
12     public:
13
14     Bit(int n){bit.resize(n+1,0);this->sz=n;}
15
16     void init(vector<ll> vec)

```

```

17 {
18     for(int i=0;i<sz;i++)
19     {
20         update(i+1,vec[i]);
21     }
22 }
23
24 void update(int p,ll val)//1i
25 {
26     while(p<=sz)
27     {
28         bit[p]+=val;
29         p+=(p&(-p));
30     }
31 }
32 ll query(int p)//[1,p]1i
33 {
34     ll ans=0;
35     while(p>0)
36     {
37         ans+=bit[p];
38         p-=(p&-p);
39     }
40     return ans;
41 }
42
43 ll rquery(ll left,ll right)//[1,r]1i
44 {
45     return query(right)-query(left-1);
46 }
47 };

```

## 6.9. sqrt

```

1 struct Sqrt
2 {
3     vector<ll> vec;
4     vector<ll> blocks;
5     int sz;
6     int szb;
7
8     Sqrt(vector<ll> v,int n)
9     {

```

```

10     sz=n;
11     szb=int(ceil(sqrt(n)));
12     vec=v;
13     blocks.assign(szb,0ll);
14     for(int i=0;i<sz;i++)
15     {
16         blocks[i/szb]+=vec[i];
17     }
18 }
19
20 ll q(int r)//[0,r)
21 {
22     ll ans=0;
23     for(int i=0;i<r/szb;i++)
24     {
25         ans+=blocks[i];
26     }
27     for(int i=(r/szb)*szb;i<r;i++)
28     {
29         ans+=vec[i];
30     }
31     return ans;
32 }
33
34 ll rq(int l,int r)//[l,r)1i
35 {
36     return q(r)-q(l-1);
37 }
38
39 void update(int pos,ll v)//0i
40 {
41     blocks[pos/szb]-=vec[pos];
42     vec[pos]=v;
43     blocks[pos/szb]+=vec[pos];
44 }
45 };

```

## 6.10. sqrtUpdateinSqrt

```

1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4

```

```

5 struct Sqrt
6 {
7     vector<ll> vec;
8     vector<ll> blocks;
9     int sz;
10    int szb;
11
12    Sqrt(vector<ll> v, int n)
13    {
14        sz = n;
15        szb = int(ceil(sqrt(n)));
16        vec = v;
17        blocks.assign(szb, 1e18);
18        for (int i = 0; i < sz; i++)
19        {
20            blocks[i / szb]=min(vec[i],blocks[i /
21                                szb]);
22        }
23
24        ll rq(int l, int r) //[l,r)0i
25        {
26            ll ans=1e18;
27            int c_l = l / szb, c_r = r / szb;
28            if (c_l == c_r)
29                for (int i = l; i <= r; i++)
30                    ans=min(ans,vec[i]);
31            else
32            {
33                for (int i = l, end = (c_l + 1) * szb -
34                    1; i <= end; i++)
35                    ans=min(ans,vec[i]);
36                for (int i = c_l + 1; i <= c_r - 1; i++)
37                    ans=min(ans,blocks[i]);
38                for (int i = c_r * szb; i <= r; i++)
39                    ans=min(ans,vec[i]);
40            }
41            return ans;
42        }
43
44        void update(int pos, ll v) // 0i
45        {
46            vec[pos] = v;
47            ll ans=1e18;

```

```

47         int ini=(pos/szb)*szb;
48         for(int i=ini;i<min(ini+szb,sz);i++)
49         {
50             ans=min(ans,vec[i]);
51         }
52         blocks[pos/szb]=ans;
53     }
54 };
55

```

## 7. Strings

### 7.1. KMP

```

1 vector<int> prefix_function(string s)
2 {
3     int n = (int)s.length();
4     vector<int> pi(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = pi[i - 1];
8         while (j > 0 && s[i] != s[j])
9             j = pi[j - 1];
10        if (s[i] == s[j])
11            j++;
12        pi[i] = j;
13    }
14    return pi;
15 }
16
17 inline void solve()
18 {
19     string texto, sub;
20     cin >> texto >> sub;
21     int n = texto.size();
22     int m = sub.size();
23     vector<int> vec = prefix_function(sub + "$" +
24                                     texto);
25     cout << count(vec.begin(), vec.end(), m) << endl
26         ;
27 }

```

### 7.2. hashing

```

1 struct StrHash
2 { // Hash polinomial con exponentes decrecientes.
3     static constexpr ll ms[] = {1'000'000'007, 1'000
4         '000'403};
5     static constexpr ll b = 500'000'000;
6     vector<ll> hs[2], bs[2];
7     StrHash(string const &s)
8     {
9         int n = s.length();
10        for(int k=0;k<2;k++)
11        {
12            hs[k].resize(n + 1), bs[k].resize(n + 1,
13                1);
14            for(int i=0;i<n;i++)
15            {
16                hs[k][i + 1] = (hs[k][i] * b + s[i])
17                    % ms[k];
18                bs[k][i + 1] = bs[k][i] * b % ms[k];
19            }
20        }
21    }
22    ll get(int idx, int len) const
23    { // Hashes en 's[idx, idx+len)'
24        ll h[2];
25        for(int k=0;k<2;k++)
26        {
27            h[k] = hs[k][idx + len] - hs[k][idx] *
28                bs[k][len] % ms[k];
29            if (h[k] < 0)
30                h[k] += ms[k];
31        }
32        return (h[0] << 32) | h[1];
33    }
34 };
35
36 //concat substrings(or strings) from non necessary
37 two differents strings [idx,idx+lex)
38 ll concat_cross_hashes(const StrHash& A, int i1, int
39     len1, const StrHash& B, int i2, int len2)
40 {
41     ll res[2];
42     for (int k = 0; k < 2; ++k)

```

```

37 {
38     // hash de substring A[i1..i1+len1-1]
39     ll h1 = A.hs[k][i1 + len1] - A.hs[k][i1] * A
40         .bs[k][len1] % A.ms[k];
41     if (h1 < 0) h1 += A.ms[k];
42
43     // hash de substring B[i2..i2+len2-1]
44     ll h2 = B.hs[k][i2 + len2] - B.hs[k][i2] * B
45         .bs[k][len2] % B.ms[k];
46     if (h2 < 0) h2 += B.ms[k];
47
48     // combinacion: h1 * b^len2 + h2
49     res[k] = (h1 * A.bs[k][len2] + h2) % A.ms[k];
50 }
51
52 //0 indexed
53 ll h=StrHash("Hola").get(0,0+"Hola".size());
54

```

### 7.3. hashing2d

```

1 struct Hashing
2 {
3     vector<vector<int>> hs;
4     vector<int> PWX, PWY;
5     int n, m;
6     static const int PX = 3731, PY = 2999, mod =
7         998244353;
8     Hashing() {}
9     Hashing(vector<string> &s)
10     {
11         n = (int)s.size(), m = (int)s[0].size();
12         hs.assign(n + 1, vector<int>(m + 1, 0));
13         PWX.assign(n + 1, 1);
14         PWY.assign(m + 1, 1);
15         for (int i = 0; i < n; i++)
16             PWX[i + 1] = 1LL * PWX[i] * PX % mod;
17         for (int i = 0; i < m; i++)
18             PWY[i + 1] = 1LL * PWY[i] * PY % mod;
19         for (int i = 0; i < n; i++)

```

```

19 {
20     for (int j = 0; j < m; j++)
21     {
22         hs[i + 1][j + 1] = s[i][j] - 'a' +
23             1;
24     }
25 }
26 for (int i = 0; i <= n; i++)
27 {
28     for (int j = 0; j < m; j++)
29     {
30         hs[i][j + 1] = (hs[i][j + 1] + 1LL *
31             hs[i][j] * PY % mod) % mod;
32     }
33 }
34 for (int i = 0; i < n; i++)
35 {
36     for (int j = 0; j <= m; j++)
37     {
38         hs[i + 1][j] = (hs[i + 1][j] + 1LL *
39             hs[i][j] * PX % mod) % mod;
40     }
41 }
42 int get_hash(int x1, int y1, int x2, int y2)
43 { // 1-indexed
44     assert(1 <= x1 && x1 <= x2 && x2 <= n);
45     assert(1 <= y1 && y1 <= y2 && y2 <= m);
46     x1--;
47     y1--;
48     int dx = x2 - x1, dy = y2 - y1;
49     return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1]
50         * PWY[dy] % mod + mod) % mod -
51         1LL * (hs[x1][y2] - 1LL * hs[x1][y1]
52         * PWY[dy] % mod + mod) % mod *
53         PWX[dx] % mod + mod) %
54         mod;
55 }
56 int get_hash()
57 {
58     return get_hash(1, 1, n, m);
59 }
60 };

```

## 7.4. hashingmersense

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct StrHash {
5     using ull = unsigned long long;
6
7     static const ull MOD = (1ULL << 61) - 1;
8
9     // Generacion de base aleatoria segura
10    static ull get_base() {
11        static ull B = 0;
12        if (B == 0) {
13            mt19937_64 rng(chrono::steady_clock::now
14                           ().time_since_epoch().count());
15            B = rng() % (MOD - 300) + 256;
16            if (B % 2 == 0) B++; // Es mejor que la
17                                // base sea impar
18        }
19        return B;
20    }
21
22    vector<ull> h, p;
23
24    // Multiplicacion modular rapida O(1) sin %
25    static inline ull mul(ull a, ull b) {
26        unsigned __int128 prod = (unsigned __int128)
27            a * b;
28        ull r = (ull)(prod & MOD) + (ull)(prod >>
29            61);
30        if (r >= MOD) r -= MOD;
31        return r;
32    }
33
34    // Suma modular rapida
35    static inline ull add(ull a, ull b) {
36        ull res = a + b;
37        if (res >= MOD) res -= MOD;
38        return res;
39    }
40
41    // Resta modular rapida
42    static inline ull sub(ull a, ull b) {
```

```
39    ull res = a - b;
40    if (a < b) res += MOD;
41    return res;
42 }
43
44 StrHash(const string& s) {
45     int n = s.size();
46     h.resize(n + 1);
47     p.resize(n + 1);
48     p[0] = 1;
49
50     ull B = get_base();
51
52     for (int i = 0; i < n; i++) {
53         h[i + 1] = add(mul(h[i], B), s[i]);
54         p[i + 1] = mul(p[i], B);
55     }
56
57     ull get(int l, int len) const {
58         return sub(h[l + len], mul(h[l], p[len]));
59     }
60 }
61 };
```

## 7.5. manacher

```
1 vector<int> manacher_odd(string s) {
2     int n = s.size();
3     s = "$" + s + "^";
4     vector<int> p(n + 2);
5     int l = 0, r = 1;
6     for (int i = 1; i <= n; i++) {
7         if (i <= r) {
8             p[i] = min(r - i, p[l + (r - i)]);
9         }
10        while (s[i - p[i]] == s[i + p[i]]) {
11            p[i]++;
12        }
13        if (i + p[i] > r) {
14            l = i - p[i], r = i + p[i];
15        }
16    }
17    return vector<int>(begin(p) + 1, end(p) - 1);
```

```

18 }
19
20 vector<int> manacher(string s) {
21     string t;
22     for(auto c: s) {
23         t += string("#") + c;
24     }
25     auto res = manacher_odd(t + "#");
26     return vector<int>(begin(res) + 1, end(res) - 1)
27     ;
28 }

```

## 7.6. zfunction

```

1 vector<int> z_function(string s) {
2     int n = s.size();
3     vector<int> z(n);
4     int l = 0, r = 0;
5     for(int i = 1; i < n; i++) {
6         if(i < r) {
7             z[i] = min(r - i, z[i - l]);
8         }
9         while(i + z[i] < n && s[z[i]] == s[i + z[i]
10             ]) {
11             z[i]++;
12         }
13         if(i + z[i] > r) {
14             l = i;
15             r = i + z[i];
16         }
17     }
18     return z;
19 }

```

## 8. Geometry

### 8.1. KDTree

```

1 double mejordist;
2 double menordis;
3
4 //P x,y

```

```

5 //dist distancia euclidana
6 //El kde tree nos puede ayudar a calcular el vecino
7 //mas cercano a un punto
8 struct KDTree
9 {
10     struct Nodo
11     {
12         P p;
13         Nodo* left;
14         Nodo* rigth;
15         int id;
16         Nodo(P pt):p(pt),left(nullptr),rigth(nullptr)
17         {}
18     };
19     Nodo* raiz;
20
21     Nodo* insertar(Nodo* nodo,P pt,int dep)
22     {
23         if(nodo==nullptr) return new Nodo(pt);
24         int dim=dep%2;
25         if(dim==0)
26         {
27             if(pt.x<nodo->p.x)nodo->left=insertar(
28                 nodo->left,pt,dep+1);
29             else nodo->rigth=insertar(nodo->rigth,pt,
30                 dep+1);
31         }
32         else
33         {
34             if(pt.y<nodo->p.y)nodo->left=insertar(
35                 nodo->left,pt,dep+1);
36             else nodo->rigth=insertar(nodo->rigth,pt,
37                 dep+1);
38         }
39         return nodo;
40     }
41
42     // void search(Nodo *nodo,P pt,int dep)
43     // {
44     //     if(nodo==nullptr) return;
45     //     ans=nodo->p;
46 }

```

```

43 //      int dim=dep%2;
44
45 //      if(dim==0)
46 //      {
47 //          if(pt.x<nodo->p.x) search(nodo->left,
48 pt,dep+1);
49 //      else search(nodo->right,pt,dep+1);
50 //      }
51 //      else
52 //      {
53 //          if(pt.y<nodo->p.y) search(nodo->left,
54 pt,dep+1);
55 //      else search(nodo->right,pt,dep+1);
56 //      }
57
58 // }
59
60 void search(Nodo* nodo,P pt,int dep,int mn=0)
61 {
62     if(nodo==nullptr) return;
63
64     double d=dist(nodo->p,pt);
65
66     if(d>mn and d<mejordist)
67     {
68         mejordist=d;
69         ans=nodo->p;
70     }
71
72     int dim=dep%2;
73
74     Nodo* first;
75     Nodo* second;
76
77     if((dim==0 && pt.x<nodo->p.x) || (dim==1 &&
78 pt.y<nodo->p.y))
79     {
80         first=nodo->left;
81         second=nodo->right;
82     }
83     else
84     {
85         first=nodo->right;

```

```

84         second=nodo->left;
85     }
86
87     search(first,pt,dep+1,mn);
88
89     double planeDist;
90
91     if(dim==0) planeDist=(pt.x-nodo->p.x)*(pt.x-
92 nodo->p.x);
93     else planeDist=(pt.y-nodo->p.y)*(pt.y-nodo->
94 p.y);
95     if(planeDist<mejordist) search(second,pt,dep
96 +1,mn);
97
98 }
99
100 KDTree():raiz(nullptr){};
101
102 };

```

## 8.2. Point

```

1 /*typedef double T;
2 typedef complex<T> pt;
3 #define x real()
4 #define y imag()*/
5
6 //typedef long long ll;
7 //typedef long double ll;
8
9 struct point
10 {
11     ll x, y;
12     point() {}
13     point(ll x, ll y): x(x), y(y) {}
14     point operator -(point p) {return point(x - p.x, y
15 - p.y);}
16     point operator +(point p) {return point(x + p.x, y
17 + p.y);}
18     ll sq() {return x * x + y * y;}
19     double abs() {return sqrt(sq());}
20     ll operator ^(point p) {return x * p.y - y * p.x;}
21     ll operator *(point p) {return x * p.x + y * p.y
22 ;}

```

```

20     point operator *(ll a) {return point(x * a, y *
21         a);}
22     bool operator <(const point& p) const {return x ==
23         p.x ? y < p.y : x < p.x;}
24     bool left(point a, point b) {return ((b - a) ^ (*
25         this - a)) >= 0;}
26     ostream& operator<<(ostream& os) {
27         return os << "(" << x << ", " << y << ")";
28     }
29 };
30 void polarSort(vector<point>& v) {
31     sort(v.begin(), v.end(), [] (point a, point b) {
32         const point origin{0, 0};
33         bool ba = a < origin, bb = b < origin;
34         if (ba != bb) { return ba < bb; }
35         return (a^b) > 0;
36     });
37 }

```

### 8.3. convexHull

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 struct pt
7 {
8     double x, y;
9     bool operator==(pt const &t) const
10     {
11         return x == t.x && y == t.y;
12     }
13 };
14
15 int orientation(pt a, pt b, pt c)
16 {
17     double v = a.x * (b.y - c.y) + b.x * (c.y - a.y) +
18         c.x * (a.y - b.y);
19     if (v < 0)
20         return -1; // clockwise

```

```

21     if (v > 0)
22         return +1; // counter-clockwise
23     return 0;
24 }
25 bool cw(pt a, pt b, pt c, bool include_collinear)
26 {
27     int o = orientation(a, b, c);
28     return o < 0 || (include_collinear && o == 0);
29 }
30 bool collinear(pt a, pt b, pt c) { return
31     orientation(a, b, c) == 0; }
32 void convex_hull(vector<pt> &a, bool
33     include_collinear = false)
34 {
35     pt p0 = *min_element(a.begin(), a.end(), [](pt a,
36         pt b)
37         { return make_pair(a.y, a.x) <
38             make_pair(b.y, b.x); });
39     sort(a.begin(), a.end(), [&p0](const pt &a, const
40         pt &b)
41         {
42             int o = orientation(p0, a, b);
43             if (o == 0)
44                 return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)
45                     *(p0.y-a.y)
46                     < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)
47                         *(p0.y-b.y);
48             return o < 0; });
49     if (include_collinear)
50     {
51         int i = (int)a.size() - 1;
52         while (i >= 0 && collinear(p0, a[i], a.back()))
53             i--;
54         reverse(a.begin() + i + 1, a.end());
55     }
56     vector<pt> st;
57     for (int i = 0; i < (int)a.size(); i++)
58     {
59         while (st.size() > 1 && !cw(st[st.size() - 2],
60             st.back(), a[i], include_collinear))
61             st.pop_back();

```

```

56     st.push_back(a[i]);
57 }
58
59 if (include_collinear == false && st.size() == 2
    && st[0] == st[1])
60     st.pop_back();
61
62 a = st;
63 }
64
65 vector<pt> a;
66
67 int main()
68 {
69     int n; cin >> n;
70     a.resize(n);
71     //leer como puntos
72     convex_hull(a, true); //true incluye colinear, false
73                             no lo hace
74 }
75

```

#### 8.4. pointinpoligon

```

1 struct pt {
2     long long x, y;
3     pt() {}
4     pt(long long _x, long long _y) : x(_x), y(_y) {}
5     pt operator+(const pt &p) const { return pt(x +
6         p.x, y + p.y); }
7     pt operator-(const pt &p) const { return pt(x -
8         p.x, y - p.y); }
9     long long cross(const pt &p) const { return x *
10        p.y - y * p.x; }
11    long long dot(const pt &p) const { return x * p.
12        x + y * p.y; }
13    long long cross(const pt &a, const pt &b) const
14        { return (a - *this).cross(b - *this); }
15    long long dot(const pt &a, const pt &b) const {
16        return (a - *this).dot(b - *this); }
17    long long sqrLen() const { return this->dot(*
18        this); }
19

```

```

12 };
13
14 bool lexComp(const pt &l, const pt &r) {
15     return l.x < r.x || (l.x == r.x && l.y < r.y);
16 }
17
18 int sgn(long long val) { return val > 0 ? 1 : (val
19     == 0 ? 0 : -1); }
20
21 vector<pt> seq;
22 pt translation;
23 int n;
24
25 bool pointInTriangle(pt a, pt b, pt c, pt point) {
26     long long s1 = abs(a.cross(b, c));
27     long long s2 = abs(point.cross(a, b)) + abs(
28         point.cross(b, c)) + abs(point.cross(c, a));
29     return s1 == s2;
30 }
31
32 void prepare(vector<pt> &points) {
33     n = points.size();
34     int pos = 0;
35     for (int i = 1; i < n; i++) {
36         if (lexComp(points[i], points[pos]))
37             pos = i;
38     }
39     rotate(points.begin(), points.begin() + pos,
40         points.end());
41
42     n--;
43     seq.resize(n);
44     for (int i = 0; i < n; i++)
45         seq[i] = points[i + 1] - points[0];
46     translation = points[0];
47 }
48
49 bool pointInConvexPolygon(pt point) {
50     point = point - translation;
51     if (seq[0].cross(point) != 0 &&
52         sgn(seq[0].cross(point)) != sgn(seq[0].
53         cross(seq[n - 1])))
54         return false;
55     if (seq[n - 1].cross(point) != 0 &&
56         sgn(seq[n - 1].cross(point)) != sgn(seq[n - 1].
57         cross(seq[0])))
58         return false;
59     return true;
60 }

```

```

52         sgn(seq[n - 1].cross(point)) != sgn(seq[
           n - 1].cross(seq[0]))))
53         return false;
54
55     if (seq[0].cross(point) == 0)
56         return seq[0].sqrLen() >= point.sqrLen();
57
58     int l = 0, r = n - 1;
59     while (r - l > 1) {
60         int mid = (l + r) / 2;
61         int pos = mid;
62         if (seq[pos].cross(point) >= 0)
63             l = mid;
64         else
65             r = mid;
66     }
67     int pos = l;
68     return pointInTriangle(seq[pos], seq[pos + 1],
69         pt(0, 0), point);

```

## 9. Primalidad

### 9.1. RabinMiller

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5
6  using u64 = uint64_t;
7  using u128 = __uint128_t;
8
9  u64 binpower(u64 base, u64 e, u64 mod) {
10     u64 result = 1;
11     base %= mod;
12     while (e) {
13         if (e & 1)
14             result = (u128)result * base % mod;
15         base = (u128)base * base % mod;
16         e >>= 1;
17     }
18     return result;

```

```

19 }
20
21 bool check_composite(u64 n, u64 a, u64 d, int s) {
22     u64 x = binpower(a, d, n);
23     if (x == 1 || x == n - 1)
24         return false;
25     for (int r = 1; r < s; r++) {
26         x = (u128)x * x % n;
27         if (x == n - 1)
28             return false;
29     }
30     return true;
31 };
32
33 bool MillerRabin(u64 n, int iter=5) { // returns
34     true if n is probably prime, else returns false.
35     cout<<iter<<endl;
36     if (n < 4)
37         return n == 2 || n == 3;
38
39     int s = 0;
40     u64 d = n - 1;
41     while ((d & 1) == 0) {
42         d >>= 1;
43         s++;
44     }
45
46     for (int i = 0; i < iter; i++) {
47         int a = 2 + rand() % (n - 3);
48         if (check_composite(n, a, d, s))
49             return false;
50     }
51     return true;
52 }
53
54 int main()
55 {
56
57     cout<<MillerRabin(100000001,30)<<endl;
58     return 0;
59 }

```

## 9.2. pollardRho

```
1 #include <iostream>
2 #include <cstdlib>
3 #include <cstdio>
4 #include <cmath>
5 #include <cassert>
6 #include <map>
7
8 using namespace std;
9
10 typedef long long ll;
11
12 #define forn(i, n) for (int i = 0; i < (int)(n); i++)
13 #define forsn(i, s, n) for (int i = int(s); i < (int)(n); i++)
14
15 // rabin miller
16
17 ll potlog(ll a, ll b, const ll M)
18 {
19     ll res = 1;
20     while (b)
21     {
22         if (b % 2)
23             res = (__int128(res) * a) % M;
24         a = (__int128(a) * a) % M;
25         b /= 2;
26     }
27     return res;
28 }
29
30 bool primo(ll n)
31 {
32     if (n < 2)
33         return false;
34     if (n == 2)
35         return true;
36     ll D = n - 1, S = 0;
37     while (D % 2 == 0)
38     {
39         D /= 2;
40         S++;
```

```
41     }
42     // n-1 = 2^S * D
43     static const int STEPS = 16;
44     forn(pasos, STEPS)
45     {
46         const ll A = 1 + rand() % (n - 1);
47         ll M = potlog(A, D, n);
48         if (M == 1 || M == (n - 1))
49             goto next;
50         forn(k, S - 1)
51         {
52             M = (__int128(M) * M) % n;
53             if (M == (n - 1))
54                 goto next;
55         }
56         return false;
57     next:;
58     }
59     return true;
60 }
61
62 // pollard's rho
63
64 ll mcd(ll a, ll b) { return (a == 0) ? b : mcd(b % a, a); }
65
66 ll factor(ll n)
67 {
68     static ll A, B;
69     A = 1 + rand() % (n - 1);
70     B = 1 + rand() % (n - 1);
71     #define f(x) ((__int128(x) * (x + B)) % n + A)
72     ll x = 2, y = 2, d = 1;
73     while (d == 1 || d == -1)
74     {
75         x = f(x);
76         y = f(f(y));
77         d = mcd(x - y, n);
78     }
79     return abs(d);
80 }
81
82 map<ll, ll> fact;
```

```

84 void factorize(ll n)
85 {
86     assert(n > 0);
87     while (n > 1 && !primo(n))
88     {
89         ll f;
90         do
91         {
92             f = factor(n);
93         } while (f == n);
94         n /= f;
95         factorize(f);
96         for (auto &it : fact)
97             while (n % it.first == 0)
98             {
99                 n /= it.first;
100                 it.second++;
101             }
102     }
103     if (n > 1)
104         fact[n]++;
105 }
106
107 int main()
108 {
109     ll N;
110     while (cin >> N && N)
111     {
112         fact.clear();
113         factorize(N);
114         for (const auto &it : fact)
115             cout << it.first << "^" << it.second <<
116                 " ";
117         cout << endl;
118     }
119     return 0;

```

## 10. Graphs

### 10.1. articulationpoint

```

1 int n; // number of nodes

```

```

2 vector<vector<int>> gf; // gfacency list of graph
3
4 vector<bool> vis;
5 vector<int> tin, low;
6 int timer;
7
8 void dfs(int v, int p = -1) {
9     vis[v] = true;
10    tin[v] = low[v] = timer++;
11    int children=0;
12    for (int to : gf[v]) {
13        if (to == p) continue;
14        if (vis[to]) {
15            low[v] = min(low[v], tin[to]);
16        } else {
17            dfs(to, v);
18            low[v] = min(low[v], low[to]);
19            if (low[to] >= tin[v] && p!=-1)
20                IS_CUTPOINT(v);
21            ++children;
22        }
23    }
24    if(p == -1 && children > 1)
25        IS_CUTPOINT(v);
26 }
27
28 void find_cutpoints() {
29     timer = 0;
30     vis.assign(n, false);
31     tin.assign(n, -1);
32     low.assign(n, -1);
33     for (int i = 0; i < n; ++i) {
34         if (!vis[i])
35             dfs (i);
36     }
37 }

```

### 10.2. bridges

```

1 void IS_BRIDGE(int v,int to); // some function to
   process the found bridge
2 int n; // number of nodes
3 vector<vector<int>> gf; // gfacency list of graph

```

```

4
5 vector<bool> vis;
6 vector<int> tin, low;
7 int timer;
8
9 void dfs(int v, int p = -1) {
10     vis[v] = true;
11     tin[v] = low[v] = timer++;
12     bool parent_skipped = false;
13     for (int to : gf[v]) {
14         if (to == p && !parent_skipped) {
15             parent_skipped = true;
16             continue;
17         }
18         if (vis[to]) {
19             low[v] = min(low[v], tin[to]);
20         } else {
21             dfs(to, v);
22             low[v] = min(low[v], low[to]);
23             if (low[to] > tin[v])
24                 IS_BRIDGE(v, to);
25         }
26     }
27 }
28
29 void find_bridges() {
30     timer = 0;
31     vis.assign(n, false);
32     tin.assign(n, -1);
33     low.assign(n, -1);
34     for (int i = 0; i < n; ++i) {
35         if (!vis[i])
36             dfs(i);
37     }
38 }

```

### 10.3. dijkstra

```

1 void dijkstra(int s)
2 {
3     set<pair<int,int>> pq;
4     dis.assign(n, INF);
5     dis[s]=0;

```

```

6     pq.insert({0,s});
7     while(!pq.empty())
8     {
9         pair<int,int> top=*pq.begin();
10        pq.erase(pq.begin());
11
12        int v=top.second;
13        int d=top.first;
14
15        for(auto [v2,cost]:gf[v])
16        {
17            if(dis[v2]>dis[v]+cost)
18            {
19                if(dis[v2]<INF)
20                {
21                    pq.erase(pq.find(make_pair(dis[
22                        v2],v2)));
23                }
24                dis[v2]=dis[v]+cost;
25                pq.insert(make_pair(dis[v2],v2));
26            }
27        }
28    }

```

### 10.4. dinic

```

1 struct FlowEdge {
2     int v, u;
3     long long cap, flow = 0;
4     FlowEdge(int v, int u, long long cap) : v(v), u(
5         u), cap(cap) {}
6 };
7
8 struct Dinic {
9     const long long flow_inf = 1e18;
10    vector<FlowEdge> edges;
11    vector<vector<int>> gf;
12    int n, m = 0;
13    int s, t;
14    vector<int> level, ptr;
15    queue<int> q;

```

```

16 Dinic(int n, int s, int t) : n(n), s(s), t(t) {
17     gf.resize(n);
18     level.resize(n);
19     ptr.resize(n);
20 }
21
22 void add_edge(int v, int u, long long cap) {
23     edges.emplace_back(v, u, cap);
24     edges.emplace_back(u, v, 0);
25     gf[v].push_back(m);
26     gf[u].push_back(m + 1);
27     m += 2;
28 }
29
30 bool bfs() {
31     while (!q.empty()) {
32         int v = q.front();
33         q.pop();
34         for (int id : gf[v]) {
35             if (edges[id].cap == edges[id].flow)
36                 continue;
37             if (level[edges[id].u] != -1)
38                 continue;
39             level[edges[id].u] = level[v] + 1;
40             q.push(edges[id].u);
41         }
42     }
43     return level[t] != -1;
44 }
45
46 long long dfs(int v, long long pushed) {
47     if (pushed == 0)
48         return 0;
49     if (v == t)
50         return pushed;
51     for (int& cid = ptr[v]; cid < (int)gf[v].
52         size(); cid++) {
53         int id = gf[v][cid];
54         int u = edges[id].u;
55         if (level[v] + 1 != level[u])
56             continue;
57         long long tr = dfs(u, min(pushed, edges[
58             id].cap - edges[id].flow));
59         if (tr == 0)

```

```

58             continue;
59             edges[id].flow += tr;
60             edges[id ^ 1].flow -= tr;
61             return tr;
62         }
63         return 0;
64     }
65
66     long long flow() {
67         long long f = 0;
68         while (true) {
69             fill(level.begin(), level.end(), -1);
70             level[s] = 0;
71             q.push(s);
72             if (!bfs())
73                 break;
74             fill(ptr.begin(), ptr.end(), 0);
75             while (long long pushed = dfs(s,
76                 flow_inf)) {
77                 f += pushed;
78             }
79             return f;
80         }
81     };

```

## 10.5. eulerianpath

```

1 int main() {
2     int n;
3     vector<vector<int>> g(n, vector<int>(n));
4     // reading the graph in the adjacency matrix
5
6     vector<int> deg(n);
7     for (int i = 0; i < n; ++i) {
8         for (int j = 0; j < n; ++j)
9             deg[i] += g[i][j];
10    }
11
12    int first = 0;
13    while (first < n && !deg[first])
14        ++first;
15    if (first == n) {

```

```

16     cout << -1;
17     return 0;
18 }
19
20 int v1 = -1, v2 = -1;
21 bool bad = false;
22 for (int i = 0; i < n; ++i) {
23     if (deg[i] & 1) {
24         if (v1 == -1)
25             v1 = i;
26         else if (v2 == -1)
27             v2 = i;
28         else
29             bad = true;
30     }
31 }
32
33 if (v1 != -1)
34     ++g[v1][v2], ++g[v2][v1];
35
36 stack<int> st;
37 st.push(first);
38 vector<int> res;
39 while (!st.empty()) {
40     int v = st.top();
41     int i;
42     for (i = 0; i < n; ++i)
43         if (g[v][i])
44             break;
45     if (i == n) {
46         res.push_back(v);
47         st.pop();
48     } else {
49         --g[v][i];
50         --g[i][v];
51         st.push(i);
52     }
53 }
54
55 if (v1 != -1) {
56     for (size_t i = 0; i + 1 < res.size(); ++i)
57         if ((res[i] == v1 && res[i + 1] == v2)
58             ||

```

```

58         (res[i] == v2 && res[i + 1] == v1))
59         {
60             vector<int> res2;
61             for (size_t j = i + 1; j < res.size()
62                 (); ++j)
63                 res2.push_back(res[j]);
64             for (size_t j = 1; j <= i; ++j)
65                 res2.push_back(res[j]);
66             res = res2;
67             break;
68         }
69     }
70     for (int i = 0; i < n; ++i) {
71         for (int j = 0; j < n; ++j) {
72             if (g[i][j])
73                 bad = true;
74         }
75     }
76
77     if (bad) {
78         cout << -1;
79     } else {
80         for (int x : res)
81             cout << x << " ";
82     }
83 }

```

## 10.6. isDag

```

1 vector<vector<int>> gf; // lista de adyacencia
2 vector<int> visited; // 0 = no visitado, 1 =
3     visitando, 2 = visitado
4
5 bool dfs(int u) {
6     visited[u] = 1; // visitando
7     for (int v : gf[u]) {
8         if (visited[v] == 1) return true; // ciclo
9         detectado
10        if (visited[v] == 0 && dfs(v)) return true;
11    }
12    visited[u] = 2; // visitado

```

```

11     return false;
12 }
13
14 bool isDAG(int n) {
15     visited.assign(n, 0);
16     for (int i = 0; i < n; i++)
17         if (visited[i] == 0 && dfs(i))
18             return false; // hay ciclo
19     return true; // no hay ciclos
20 }

```

## 10.7. isDagv2

```

1 vector<vector<int>> gf;
2 vector<bool> vis;
3 set<int> st;
4 bool sw = 1;
5 int n;
6
7 void isDAG(int nodo)
8 {
9     // no olvidar recorrer con un for todo
10    //ya que no siempre estan conectados
11    if (!sw) return;
12    vis[nodo] = 1;
13    st.insert(nodo);
14    for (auto hijo : gf[nodo])
15    {
16        if (st.count(hijo) == 1)
17        {
18            sw = 0;
19            return;
20        }
21        if (!vis[hijo])
22        {
23            vis[hijo] = 1;
24            isDAG(hijo);
25        }
26    }
27    st.erase(nodo);
28 }

```

## 10.8. khunMaxbipartiteMatching

```

1 //aca a la hora de aniadir un edge, debemos ya tener
  //que a es el grupo 1 y b el grupo 2
2
3 struct Kuhn {
4     int n, m;
5     vector<vector<int>> adj;
6     vector<int> mt, vis;
7
8     Kuhn(int n, int m) : n(n), m(m) {
9         adj.resize(n);
10        mt.assign(m, -1);
11        vis.assign(n, 0);
12    }
13
14    void addEdge(int u, int v) {
15        adj[u].push_back(v);
16    }
17
18    bool dfs(int u) {
19        if (vis[u]) return false;
20        vis[u] = 1;
21
22        for (int v : adj[u]) {
23            if (mt[v] == -1 || dfs(mt[v])) {
24                mt[v] = u;
25                return true;
26            }
27        }
28
29        return false;
30    }
31
32    int maximum_matching() {
33        int ans = 0;
34
35        for (int u = 0; u < n; ++u) {
36            fill(vis.begin(), vis.end(), 0);
37            ans += dfs(u);
38        }
39
40        return ans;
41    }

```

```
42 |};
```

## 10.9. lcabinaryliftin

```
1 | int n, l;  
2 | vector<vector<int>> gf;  
3 |  
4 | int timer;  
5 | vector<int> tin, tout;  
6 | vector<vector<int>> up;  
7 |  
8 | void dfs(int v, int p)  
9 | {  
10 |     tin[v] = ++timer;  
11 |     up[v][0] = p;  
12 |     for (int i = 1; i <= l; ++i)  
13 |         up[v][i] = up[up[v][i-1]][i-1];  
14 |  
15 |     for (int u : gf[v]) {  
16 |         if (u != p)  
17 |             dfs(u, v);  
18 |     }  
19 |  
20 |     tout[v] = ++timer;  
21 | }  
22 |  
23 | bool is_ancestor(int u, int v)  
24 | {  
25 |     return tin[u] <= tin[v] && tout[u] >= tout[v];  
26 | }  
27 |  
28 | int lca(int u, int v)  
29 | {  
30 |     if (is_ancestor(u, v))  
31 |         return u;  
32 |     if (is_ancestor(v, u))  
33 |         return v;  
34 |     for (int i = l; i >= 0; --i) {  
35 |         if (!is_ancestor(up[u][i], v))  
36 |             u = up[u][i];  
37 |     }  
38 |     return up[u][0];  
39 | }
```

```
40 |  
41 | void preprocess(int root) {  
42 |     tin.resize(n);  
43 |     tout.resize(n);  
44 |     timer = 0;  
45 |     l = ceil(log2(n));  
46 |     up.assign(n, vector<int>(l + 1));  
47 |     dfs(root, root);  
48 | }
```

## 10.10. toposort

```
1 | #include <bits/stdc++.h>  
2 | using namespace std;  
3 | int n; // number of vertices  
4 | vector<vector<int>> gf;  
5 | vector<bool> vis;  
6 | vector<int> ans;  
7 |  
8 | void dfs(int v) { // 0i  
9 |     vis[v] = true;  
10 |     for (int u : gf[v]) {  
11 |         if (!vis[u]) {  
12 |             dfs(u);  
13 |         }  
14 |     }  
15 |     ans.push_back(v);  
16 | }  
17 |  
18 | void topological_sort() {  
19 |     vis.assign(n, false);  
20 |     ans.clear();  
21 |     for (int i = 0; i < n; ++i) {  
22 |         if (!vis[i]) {  
23 |             dfs(i);  
24 |         }  
25 |     }  
26 |     reverse(ans.begin(), ans.end());  
27 | }
```

## 11. Overloads

### 11.1. pq

```
1 struct cmp
2 {
3     //mayor tiene prioridad
4     bool operator()(const int& a, const int& b)
5         const {
6             return a<b;
7         }
8 };
9 //priority_queue<Data,vector<Data>,cmp> pq
```

### 11.2. set

```
1 struct cmpST
2 {
3     //de menor a mayor
4     bool operator()(const int &a,const int &b)
5     {
6         return a<b;
7     }
8 };
9
10 //set<int,cmpST> st;
```

## 12. Dp

### 12.1. covexHullTrick

```
1 struct ConvexHull
2 {
3     struct Ln
4     {
5         ll m,b;//m pendiente m1>m2>m3... b constante
6     };
7
8     vector<Ln> cht;
```

```
9     bool tapa(Ln l1, Ln l2, Ln l3)//verifica si con
10     al agregar la linea 3 la 2 se vuelve
11     irrelevante
12 {
13     return (__int128)(l3.m-l2.m)*(l1.b-l3.b)>=(
14         __int128)(l2.b-l3.b)*(l3.m-l1.m);
15     //<= para maximos
16 }
17 void addLine(Ln nueva)
18 {
19     int sz=cht.size();
20     while(cht.size()>1)
21     {
22         sz=cht.size();
23         if(tapa(cht[sz-2],cht[sz-1],nueva))cht.
24             pop_back();
25         else break;
26     }
27     cht.push_back(nueva);
28 }
29
30 ll f(int p,ll x)
31 {
32     return cht[p].m*x+cht[p].b;
33 }
34
35 int query(ll x)//te da el indice de la mejor
36 linea
37 {
38     int l = 0, r = cht.size() - 1;
39
40     auto f = [&](int i)
41     {
42         return cht[i].m * x + cht[i].b;
43     };
44
45     while(l < r)
46     {
47         int mid = (l + r) / 2;
48         //>= para maximos
49         if(f(mid) <= f(mid + 1))r = mid;
50         else l = mid + 1;
51     }
52     return l;
```

```
48 |     }  
49 | };
```

## 12.2. list

```
1 int lis(vector<int> const& a) {  
2     int n = a.size();  
3     const int INF = 1e9;  
4     vector<int> d(n+1, INF);  
5     d[0] = -INF;  
6  
7     for (int i = 0; i < n; i++) {  
8         int l = upper_bound(d.begin(), d.end(), a[i]  
9             ]) - d.begin();  
10        if (d[l-1] < a[i] && a[i] < d[l])  
11            d[l] = a[i];  
12    }  
13  
14    int ans = 0;  
15    for (int l = 0; l <= n; l++) {  
16        if (d[l] < INF)  
17            ans = l;  
18    }  
19    return ans;  
20 }
```